

SIEMENS EDA

Catapult[®] Synthesis Release Notes

Software Version 2025.2_1
June 2025

SIEMENS

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this Documentation shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: www.plm.automation.siemens.com/global/en/legal/trademarks.html. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center: support.sw.siemens.com

Send Feedback on Documentation: support.sw.siemens.com/doc_feedback_form

Table of Contents

Introduction.....	2
Platform Compatibility.....	4
License Daemon Requirements.....	5
Software Compatibility.....	6
Supported FPGA Technologies.....	10
Backwards Compatibility.....	13
Removing unused DirectInput Ports (2024.1).....	13
MAP_TO_MODULE syntax for CCORES (2023.1).....	13
mgc_ioport Library (10.6).....	14
New commands for hierarchical blocks (10.5).....	14
Release 2025.2_1.....	15
Corrected Problems.....	15
Release 2025.2.....	16
Enhancements/Changes.....	16
<i>Synchronization of Explicit Objects in SystemC (ac_sync)</i>	16
<i>SystemC Support for Non-Blocking Peek</i>	18
<i>Automatic Dual-Port/N-Port Wrapper Generation for SinglePort RAMs</i>	20
<i>Passing ROM through Hierarchy</i>	20
<i>HLS IP Enhancements</i>	21
<i>Improvements for inferring Reset in SystemC Designs</i>	21
Changes to Directives/Options/Commands/Pragmas.....	22
<i>Directive Changes</i>	22
<i>Option Changes</i>	23
<i>Pragma Changes</i>	24
<i>Command Changes</i>	24
Corrected Problems.....	24
<i>Catapult</i>	24
Release 2025.1_1.....	27
Enhancements.....	27
<i>CLUSTER_INPUT_REGISTERS/ CLUSTER_OUTPUT_REGISTERS directives added</i>	27
Corrected Problems.....	27
Release 2025.1.....	29
Enhancements/Changes.....	29
<i>Scope-based CCORES</i>	29

Table of Contents

<i>Advanced Datapath Optimization enhancements (Beta)</i>	30
<i>Broadcast AC Channel to multiple receivers on same channel more efficiently</i>	31
<i>Redundant RTL Code Elimination</i>	32
<i>Memory/FIFO Generator Enhancements</i>	33
<i>Catapult Library Changes</i>	33
<i>Cross-probe 'analyze' schematic in Design Analyzer (alpha)</i>	33
<i>Source Code C-Step Explorer in Design Analyzer (alpha)</i>	34
<i>HLS IP Enhancements</i>	35
<i>Parallel CCOREs (alpha)</i>	35
<i>Constraint Sweeping using YAML files (alpha)</i>	36
Changes to Directives/Options/Commands/Pragmas.....	36
<i>Directive Changes</i>	36
<i>Option Changes</i>	37
<i>Command Changes</i>	37
<i>Pragma Changes</i>	38
Corrected Problems.....	38
<i>Catapult</i>	38
<i>CCoverCheck</i>	41
<i>CDesignChecker</i>	41
<i>CFormalAssert</i>	42
<i>CFormalSLEC</i>	42
Release 2024.2_2	43
License Server Update 11.19.01 to 11.19.4.1.....	43
Corrected Problems.....	44
Release 2024.2_1	45
Corrected Problems.....	45
Release 2024.2	47
Enhancements/Changes.....	47
<i>New DataPath Operator Optimizer</i>	47
<i>Automatic Splitting of Struct/Array datatype Interface ports</i>	48
<i>Non-Blocking Peek (Beta)</i>	50
<i>IDLE signal logic generation for SystemC designs</i>	51
<i>New Branch View in Design Analyzer</i>	52
<i>Value Range analysis with ac_int</i>	52
<i>Mutually Exclusive sharing for multi-port memories</i>	53
<i>Automatic smoothing of ASIC Library Data</i>	54
Added support for the AprisaRTL flow (Beta).....	55
<i>Catapult Library Changes</i>	55
Added RTL Manifest flow.....	55

Table of Contents

Changes to Directives/Options/Commands.....	57
<i>Directive Changes</i>	57
<i>Option Changes</i>	57
<i>Command Changes</i>	59
<i>Pragma Changes</i>	59
Corrected Problems.....	59
Release 2024.1_3.....	65
Corrected Problems.....	65
Release 2024.1_2.....	66
Corrected Problems.....	66
Release 2024.1_1.....	68
Enhancements/Changes.....	68
Corrected Problems.....	68
Release 2024.1.....	70
Enhancements/Changes.....	70
<i>Improved Logic Optimization</i>	70
<i>Improved Timing Estimates for chained Arithmetic Logic</i>	71
<i>New class for Array Handling (ac_bank_array)</i>	71
<i>Support for HLS4ML flow (beta)</i>	72
<i>Flushing Support for AC_WINDOW_V2</i>	72
<i>Shared Memory Port Reservation</i>	72
<i>ASIC Library Enhancements</i>	74
Changes to Directives/Options/Commands.....	75
<i>Directive Changes</i>	75
<i>Option Changes</i>	76
<i>Command Changes</i>	77
Corrected Problems.....	77
Release 2023.2_2.....	83
Enhancements/Changes.....	83
Corrected Problems.....	83
Release 2023.2_1.....	85
Corrected Problems.....	85
Release 2023.2.....	86
Enhancements/Changes.....	86
<i>Improved Struct Type Interface Mapping</i>	86
<i>New Timing Model for Reporting</i>	87
<i>Value Range Analysis Tool</i>	88
<i>Shared Memories using ac_sync</i>	88
<i>AC Window 2.0</i>	90

Table of Contents

<i>DirectOutput Resource Type</i>	90
<i>Design Analyzer Improvements</i>	91
<i>Library Changes</i>	91
<i>New CFormal Assert App (Beta)</i>	91
<i>Bus Slave Generator Flow (Beta)</i>	92
<i>Punch ac::probe() Signals to Top-Level (Beta)</i>	92
Changes to Directives/Options/Commands.....	93
<i>Directive Changes</i>	93
<i>Option Changes</i>	94
<i>Command Changes</i>	95
Corrected Problems.....	95
Known Problems and Workarounds	100
Color Space Conversion example uses unavailable library (2025.1).....	100
Mapped Bottom-Up flow in Beta.....	100
Incomplete Floating-Point, Low-Power VHDL sim model in DesignWare.....	100
SCCOM error with MSIM_AC_TYPES enabled.....	101
Running Design Analyzer on Unsupported RHEL releases.....	101
VHDL memory-based pipe may not synthesize in Design Compiler.....	102
Setting different FIFO_DEPTH values on multiple instances design.....	102
ASIC library template for mgc_div in 2022.1 may cause flow issues.....	103
Memory Generator does not merge multiple read or write ports when memory uses different read and write clocks.....	104
Memories in Bottom-Up Blocks may not be blackboxed in Low Power flow.....	104
SystemC Direct pinassocs use hierarchical name.....	104
SCVerify fails to detect simulation deadlock in bottom-up hierarchy designs.....	105
Memories with EXTERNAL pinassoc can not be shared.....	105
SystemC - Per-file Compile Flags not Passed to SCVerify.....	105
Multiple Drivers in Modular I/Os.....	105
SystemC SCVerify Limitation.....	106
ROM Inferencing.....	106
RAM Selection Algorithm May Not Choose Smallest RAM.....	106
Compact Gantt Chart is Slow for Large Designs.....	106
Direct Input Mapping Only Works for One Direct Input.....	106
The “!” Operator is Not Supported with sc_fixed Datatypes.....	107
Area Over-Estimation for Designs with Small Muxes.....	107
Longer Schedule for Loops with Multiple Exits.....	107
SCVerify Flow: Avoid Using Output Memories When Pipelining.....	107
Third-Party Information	108

Introduction

These release notes apply to the Catapult® Synthesis line of products including the C Design Checker and Library Builder (LB).

- **Catapult HLS** is targeted for designing subsystems from timed SystemC or untimed C++ code. Catapult HLS enables subsystem design through multi-level hierarchy support and channel synthesis.
- **Catapult Ultra** extends the low power performance of designs created in *Catapult HLS* by integrating the RTL low power optimization capabilities directly into the Catapult design flow. This version of Catapult also supports multi-voltage threshold libraries and on-the-fly characterization.

The Catapult® high-level synthesis tool empowers designers to use industry standard ANSI C++ and SystemC to describe functional intent, and move up to a more productive abstraction level. From these high-level descriptions Catapult generates production quality RTL. With this approach, full hierarchical systems comprised of both control blocks and algorithmic units are implemented automatically, eliminating the typical coding errors and bugs introduced by manual flows. By speeding time to RTL and automating the generation of bug free RTL, the Catapult significantly reduces the time to verified RTL. Catapult's unified flow for modeling, synthesizing, and verifying complex ASICs and FPGAs allows hardware designers to fully explore micro-architecture and interface options. Advanced power optimizations automatically provide significant reductions in dynamic power consumption. The highly interactive Catapult work flow provides full visibility and control of the synthesis process, enabling designers to rapidly converge upon the best implementation for performance, area, and power.

The following table summarizes the differences between the Catapult product options.

Feature	Catapult HLS (Part: 266066)	Catapult Ultra (Part: 266068)
C/C++/SystemC Language support	X	X
SC Verify Flow support	X	X
Clustering flow	X	X
Catapult Design Checker	X	X
Library Builder: static library characterization	X	X
Power Estimation, Optimization, and Reporting		X
Clock Gating Optimization		X
Multi-point CCOREs		X
Advanced Logic Synthesis		X
Advanced Datapath Optimization		X
Scheduler Retiming		X
Redundant RTL Code elimination		X
Automatic convert shift register to circular buffers		X
Non Power-of-two optimization enhancements		X
Analyze timing using multiple delay arcs		X
on-the-fly library characterization		X

Multi-Voltage Threshold libraries and CCOREs		X
ECO/Incremental		X

NOTE: The "Catapult HLS AMD" product option is a 10-license bundle of Catapult HLS that can only target Xilinx technologies. Since it only handles Xilinx FPGAs, it does not include any ASIC Library Builder licenses.

NOTE: The "Catapult Formal" product option includes Catapult Design Checker and the Catapult Formal apps (eg. idle, stall, and 'ignore memory precedence' checkers).

Platform Compatibility

By default, Catapult compiles the C/C++ design in a 32-bit compatibility mode. The 64-bit Catapult binary will generate RTL with 32-bits for pointers. RTL simulation environments will also need to be setup to run in 32-bit mode. In Catapult, the *Input/TargetPlatform* option controls the default platform used during compile (x86 or x86_64). Compilers used by Catapult and downstream tools, such as QuestaSIM, must be consistent with each other about which mode they compile and simulate in, 32- or 64-bit. If 32-bit mode is used, the 32-bit libraries must be available for the assembler and linker.

Your system may require one or more operating system patches installed before or after you install Catapult software. The 64-bit version of Catapult requires 64-bit Linux on running on a 64-bit CPU.

Beginning in 2023.2, for supported Linux distributions, Catapult only supports the latest minor version of a Major Version release (eg. Rocky 8.10). Previous minor versions will no longer be supported. Once the Maintenance Support date for the Major Version of a supported Linux distribution has passed, Catapult will no longer support this Major Version release. The currently supported operating systems are:

- **Rocky Linux 8.10** (Rocky Linux plans to "end of life" the maintenance of the this release in May 2029).

NOTE: The 2024.1 release was the last major Catapult version to support Red Hat 7.

NOTE: The 10.5 release was the last major Catapult version to support Red Hat 6.

NOTE: The 10.3 release was the last major Catapult version to support Windows 10.

License Daemon Requirements

This section provides the License Daemon requirements for the current release. Catapult supports the *saltd* license daemon. Although this release requires a *saltd* daemon version of 11.19.0.1 or later, **it is highly recommended that you use *saltd* 11.19.4.1 or later**. The *saltd* daemon is compatible with all supported software releases. The most recent licensing packages, based on the v4.2 *Siemens Licensing Server* releases, can be used in the current Catapult install tree or downloaded from SupportCenter (<https://support.sw.siemens.com/>).

```
$MGC_HOME/pkgs/FlexNet-11-19-5/<os>/
```

NOTE: If your license file currently points to a *mgcld* license daemon, you can update the *DAEMON* line in your license file to point to the *saltd* license daemon and it will continue to process the existing license file.

The following procedure summarizes a method for upgrading your license server:

1. Install the current release on the workstation running your license server.

NOTE: Renaming your old package and copying the new one in its place or re-linking it will ensure that all your current paths (scripts, license file, *\$PATH*) will continue to resolve.

2. Check the license file to ensure that the *DAEMON* line points to the correct copy of *saltd*.
3. Stop and restart the license server, being sure to use the correct copy of the daemon.

```
lmdown -c <path_to_license_file>  
lmgrd -c <path_to_license_file> -log <path_to_log_file>
```

You can identify the new *saltd* by invoking it from a UNIX shell with the '-v' switch to check the version. The required version is that of the client tree or newer.

The following table lists the composites included with Catapult HLS and Ultra:

Composite Name	Quantity	Composite Description	Catapult HLS (Part: 266066)	Catapult Ultra (Part: 266068)
CatapultHLS_c	1	Catapult HLS features	X	
CatapultUltra_c	1	Catapult Ultra features		X
CatapultVis_c	1	Design Analyzer	X	X
CatapultCPC_c	1	C Design Checker	X	X
catapultlb_c	1	Library Builder GUI	X	X
catapultlbfarm_c	50	Library Characterization Batch Jobs	X	X
qdbg3p	1	Questa Waveform/Coverage Viewer		X

Software Compatibility

The following table denotes the supported versions of third party software and compilers for each operating system for the latest Catapult release. The SystemC library and Make utility are only needed for the Catapult SCVerify flow. Versions in a bold font denotes version updates in the latest release (compared to the last release). For a historical view of these changes, there is a list of software compatibility changes for the last two major release cycles after the table.

Software	Vendor	Version(s)	Notes
Synthesis Tools			
Design Compiler	Synopsys	2022.03-SP3	Libraries must be characterized with Catapult Library Builder for version of DC being used
Oasys-RTL	Siemens	2021.1-s001	An integrated version is shipped with Catapult
Precision® RTL	Siemens	2024.2_0.4	An integrated version is shipped with Catapult
Quartus (Standard)	Altera	24.1.0.1077	If you are using newer versions of Quartus, refer to Potential Errors when compiling Quartus libraries for simulation.
Quartus (Pro)	Altera	24.1.0.115	
Radiant	Lattice	2023.2	
Synplify Pro	Synopsys	N-2017.09 SP1	
Vivado	Xilinx	2023.2	Can not be used for CCORE/Cluster estimation.
Simulation Tools			
Catapult Coverage (CCOV)	Siemens	2025.2	
CCoverCheck	Siemens	2025.1_1	Shipped in the Catapult install tree.
MATLAB/Simulink	Mathworks	R2022a (32-bit)	If you are using the previously supported version of Matlab, you must change the default compiler in Catapult for 10.3 to 4.9.2: options set Input/Compiler gcc_4.9.2
Novas/Verdi tools	Synopsys	U-2023.03-SP1	Required only for FSDB output generation from NCSim and ModelSim/Questa. If generating this file for LowPower flow outside of SCVerify, refer to PowerPro release notes for latest FSDB version.
OSCI		2.3.3	
QuestaSim®	Siemens	2024.3	If you are using Catapult Ultra / PowerPro with VHDL netlists, refer to Using QuestaSim QWAVE within Low Power Flow.
Questa MVC	Siemens	2024.3	Not directly used by any Catapult flow
VCS-MX	Synopsys	U-2023.03-SP2	
Visualizer	Siemens	2024.3	Required to read QWAVE files in Catapult Ultra

Xcelium	Cadence	22.09	Support for this tool is in Beta.
Linting Tool			
Catapult C Design Checker	Siemens	2025.1_1	Catapult supports the release version that is integrated into the Catapult install tree.
Spy Glass Lint	Synopsys	2023.03-SP1	
Power Optimization/Reporting			
PowerPro	Siemens	2025.1	Catapult supports the release version that is integrated into the Catapult install tree.
Formal Verification Tool			
Catapult Formal	Siemens	2025.1_1	Catapult supports the release version that is integrated into the Catapult install tree.
SLECPro	Siemens	2025.1	Catapult supports the release version that is integrated into the Catapult install tree.
SystemC/C++ Compilers and Debuggers			
Compiler		GCC 10.3.0	GCC version 4.9.2 and 10.3.0 are shipped with Catapult and installed in the Mgc_home tree.
Debugger		GNU gdb (GDB) 13.2	GDB is shipped with Catapult and installed in the Mgc_home tree.
File Formats			
C/C++		C++11	Catapult defaults to C++11. Most synthesizable constructs in C++14 and C++17 are also supported.
SystemC library		2.3.3	Shipped and installed with the Catapult software.
SystemC TLM headers		2.0	Shipped and installed with the Catapult software.
VHDL		1076-1993	
Verilog/		1364-2001	
SystemVerilog		1800-2009	Only used by netlister when writing verilog structs in RTL netlist
UVMf		2023.4_2	Used in UVM flow
Liberty Library			Used by Library Builder, DataPath Optimization, Low Power Optimization, and Memory Generator
QWave		2024.3	If you get an error when reading the Qwave file during the power stage, refer to Using QuestaSim QWAVE within Low Power Flow
FSDB		S, T, and U	FSDB files can be read in the low power flow and generated in the Catapult Formal flow.
Open Source Software libraries			
AC Math AC DSP AC IPL	Open Source	3.8.1	Algorithmic C (AC) IP libraries

AC Comm			
AC Sim Util		1.7.0	AC simulation utilities (eg. SCVerify, Assert)
AC Datatypes	Open Source	5.1.2	Algorithmic C (AC) Datatype (eg. ac_int, ac_fixed)
MatchLib Connections	Open Source	2.3.0	SystemC connections library

Unless noted otherwise, in each release, Catapult Formal, CDesignChecker, CCoverCheck, and Catapult Coverage are updated to the same version as Catapult.

In the previous three major Catapult releases, the following changes were made to the previous table:

- (2025.2) PowerPro/SLECPro updated from 2024.2_1 to 2025.1
- (2025.2) QuestaSIM updated from 2023.2 to 2024.3
- (2025.2) Quartus updated from 22.X to 24.1
- (2025.2) Novas/Verdi updated from T-2022.06-SP2-1 to U-2023.03-SP1
- (2025.1) Precision updated from 2024.1.0.20 to 2024.2_0.4
- (2025.1) PowerPro/SLECPro changed from 2024.1_3 to 2024.2_1
- (2024.2_1) Quartus updated from 18.0.0.614 to 22.1.1.917 (standard) and 19.3.0.222 to 22.3.0.104 (pro)
- (2024.2_1) Lattice Radiant updated from 3.0 to 2023.2
- (2024.2_1) Novas/Veridi updated from R-2020.12 to T-2022.06-SP2-1
- (2024.2) QuestaSIM updated from 2021.3 to 2023.2
- (2024.2) Xilinx Vivado changed from 2020.2 to 2023.2
- (2024.2) PowerPro/SLECPro changed from 2023.2_6 to 2024.1_3
- (2024.2) Precision RTL changed from 2023.2.0.16 to 2024.1.0.20
- (2024.1_2) Catapult Coverage changed from 2024.1 to 2024.1_1
- (2024.1_2) Catapult Formal changed from 2024.1 to 2024.1_4
- (2024.1_2) PowerPro and SLECPro changed from 2023.2_3 to 2023.2_6
- (2024.1) Spyglass Lint changed from 2020.03-SP1 to 2023.03-SP1.
- (2024.1) Precision changed from 2023.1.1.18 to 2023.2.0.16

- (2024.1) Xcelium changed from 19.09 to 22.09.
- (2023.2_1) GDB updated from 10.2 to 13.2
- (2023.2) VCS changed from R-2020.12-SP1 to U-2023.03.
- (2023.2) SystemVerilog 2009 support was added for netlisting structs on the interface.
- (2023.2) Precision RTL changed from 2022.2.0.9 to 2023.1.1.18

The latest version of Catapult Formal and C DesignChecker are always integrated into the Catapult install tree. The supported Catapult Coverage version (separate install tree) is aligned with the similarly named (eg. 2024.1) version of Catapult.

Potential Errors when compiling Quartus libraries for simulation

Due to errors when compiling the latest Quartus simulation libraries with VCS and Questa, Catapult supports the 22.1 (std) and 22.3 (pro) versions of Quartus. If you attempt to compile later versions of the Quartus simulation library with VCS 2023.03, you may get an encryption error similar to the following:

```
Error: Parsing design file '/tools/altera/23.1./quartus/eda/sim_lib/...'
Error: Error-[DECSPE] Decryption failed
Error: Error while decrypting source file
```

The impacted families for the VCS encryption issue are Stratix V, Arria V, Arria V GZ, Cyclone V, and Stratix IV.

If you use newer than the supported versions of Quartus (eg. 23.1) with Questa, you may encounter missing modules in the precompiled library (PCL) or missing dependencies when compiling for simulation outside of the PCL flow.

Using pre-2023.3 QuestaSim versions (without Rocky 8 support)

In the June 2024, RedHat ended maintenance support for RedHat7. As a result, Catapult 2024.2 only supports Rocky8.10. If you are running Catapult with an pre-2023.3 version of QuestaSim that did not support Rocky8, then you can set the following undocumented environment variable to disable OS check during QuestaSim invocation.

```
setenv MTI_BYPASS_SC_PLATFORM_CHECK 1
```

This environment variable is automatically set as part of the SCVerify flow.

Using QuestaSim QWAVE within Low Power Flow

In the Low Power flow, when Catapult reads the QWAVE switching activity file during the *power* stage, if you encounter a *read_qwave* error, related to finding the top-level instances, then you should revert to using Questa/Visualizer 2021.3. This issue is typically due to conflicts when aligning the hierarchy between the vopt-generated *design.bin* file and the QWAVE switching file or null Qwave data.

Supported FPGA Technologies

The following table shows the available Xilinx FPGA libraries for the latest release.

Xilinx Family	Speed Grades	Precision	Synplify	Vivado
Artix 7	-1, -2, -2L, -3	X	X	X
Kintex 7	-1, -2L, -3	X	X	X
Kintex UltraScale	-1, -1L, -1LV, -2, -3	X	X	X
Kintex Ultrascale+	-1, -1L, -1LV, -2, -2L, -2LV, -3	X	X	X
Versal AI Core	-1LHP, -1LP, -1MP, -2HP, -2LP, -2MP, -3HP	X	X	X
Versal HBM	-1LHP, -1LP, -1MP, -2HP, -2LP, -2MP, -3HP	X	X	X
Versal Prime	-1LP, -1MP, -2HLP, -2LP, -2MHP, -2MP, -3HP	X	X	X
Virtex 5QV (QPro)	-1		X	
Virtex 7	-1, -2, -2L, -3	X	X	X
Virtex UltraScale	-1, -2, -3	X	X	X
Virtex Ultrascale+	-1, -2, -2L, -2LV, -3	X	X	X
Virtex UltraScale+ 58G	-1, -2LV, -2L, -3			
Zynq	-1, -2, -3	X	X	X
Zynq Ultrascale+	-1, -1L, -1LV, -2, -2L, -2LV, -3	X	X	X
Zynq Ultrascale+ RFSOC	-1, -1L, -1LV, -2, -2L, -2LV	X	X	X

The following table shows the supported Altera FPGA libraries for the latest release.

Altera Family	Speed Grades	Precision	Synplify	Quartus
Agilex	-1, -2, -3, -4	X	X	X
Arria V	-3, -4, -5, -6	X	X	X
Arria V GZ	-3, -4	X	X	X
Arria 10	-1, -2, -3	X	X	X
Cyclone V	-6, -7, -8	X	X	X
Cyclone 10 GX	-5, -6	X	X	X
Stratix IV	-1, -2X, -3, -4	X	X	X
Stratix V	-1, -2, -3, -4	X	X	X

Altera Family	Speed Grades	Precision	Synplify	Quartus
Stratix 10	-1, -2, -3	X	X	X

The following table shows the supported Microsemi FPGA libraries for the latest release.

Microsemi Family	Speed Grades	Precision	Synplify
AXCELERATOR	-1, -2, STD	X	X
FUSION	-1, -2, STD	X	X
IGLOO	STD	X	X
IGLOO+	STD	X	X
IGLOO2	-1, STD	X	X
IGLOOE	STD	X	X
POLARFIRE	-1, STD	X	X
RTG4	-1, STD	X	X
SMARTFUSION	-1, STD	X	X
SMARTFUSION2	-1, STD	X	X

The following table shows the supported Achronix FPGA libraries for the latest release.

Achronix Family	Speed Grades
SPEEDSTER 16T	C1, C2
SPEEDSTER7TMLP	C1, C2
SPEEDSTER7T	C1, C2

The following table shows the supported Menta FPGA libraries for the latest release.

Menta Family	Speed Grades	Origami
ts28hpcphdgp	Typical	X

The following table shows the supported Lattice FPGA libraries for the latest release.

Lattice Family	Speed Grades	Radiant
CetrusPro-NX (LFCPNX)	-7_High-Performance_1.0V -8_High-Performance_1.0V -9_High-Performance_1.0V -8_Low-Power_1.0V -7_Low-Power_1.0V -9_Low-Power_1.0V	X
CrossLink-NX (LIFCL)	-7_High-Performance_1.0V -8_High-Performance_1.0V -9_High-Performance_1.0V -8_Low-Power_1.0V -7_Low-Power_1.0V	X

Lattice Family	Speed Grades	Radiant
	-9_Low-Power_1.0V	
Avant-E (LAV-AT)	-1, -2, -3	X

The following table shows the supported nanoXplore FPGA libraries for the latest release.

NanoXplore Family	Speed Grades	Precision
NG-ULTRA	-2	X

Backwards Compatibility

As Catapult continues to provide new functionality in each release, this section will provide a list of changes that may affect compatibility with previous releases. The intent is to allow users to continue to use their existing scripts with the new releases until they are able to migrate. Any directives and options that were provided for backwards compatibility will be deprecated after two years.

The following sections describes known changes that affect backwards compatibility. The value in parenthesis denotes the release when the change occurred.

- Removing unused DirectInput Ports (2024.1)
- MAP_TO_MODULE syntax for CCORES (2023.1)
- mgc_ioport Library (10.6)
- New commands for hierarchical blocks (10.5)

Removing unused DirectInput Ports (2024.1)

In the 2024.1 release, Catapult changed how it handles unused DirectInput Ports. Prior to 2024.1, Catapult preserved unused DirectInput Ports. In the 2024.1 release, these types of ports were optimized away.

In 2024.1_1, by default, unused DirectInput Ports are preserved (restoring the pre-2024.1 functionality). This release also introduced a new REMOVE_UNUSED_DIRECT_INPUTS directive to provide users with direct control over how Catapult handles this situation. If the directive is changed to *true*, any unused DirectInput ports will be optimized away.

```
directive set /REMOVE_UNUSED_DIRECT_INPUTS true
```

By default, this directive is set to *false*.

MAP_TO_MODULE syntax for CCORES (2023.1)

The legacy MAP_TO_MODULE syntax used in regular Bottom-Up CCORE flow is deprecated and following example illustrates the expected syntax:

Deprecated syntax

```
directive set /top_module/CCORE_module -MAP_TO_MODULE libraries.CCORE_module
```

Proper/Expected Syntax:

```
directive set /top_module/CCORE_module -MAP_TO_MODULE {[CCORE] CCORE_module.v1}
```

(.v1 is just an example and the user can choose any compatible solution added to library)

mgc_ioport Library (10.6)

Starting in 10.6, the legacy `mgc_ioport` library will no longer be enhanced. Designs using this library will not be supported in the new HLS Verification methodology. Since the `ccs_ioport` library components can replace the `mgc_ioport` components, the `mgc_ioport` library will be removed from the Catapult install tree in a future major release (at least two years after the 10.6 release date).

In the 10.1 release, Catapult introduced the `ccs_ioport` library which provided a simple, consistent naming convention for data and handshaking (rdy/vld) ports. Since its introduction, this library has been enhanced with numerous additional components to facilitate push interfaces, builtin pipes (FIFO), and FPGA high frequency interfaces. This library uses the same methodology for mapping IO resources to components as the `mgc_ioport` library but utilizes a more straight-forward naming convention.

The following table lists the `mgc_ioport` components and its `ccs_ioport` counterpart.

Port Direction	mgc_ioport Component	ccs_ioport Component
in	<code>mgc_in_wire</code>	<code>ccs_in</code>
	<code>mgc_in_wire_en</code>	<code>ccs_in_rdy</code>
	NA	<code>ccs_in_vld</code>
	<code>mgc_in_wire_wait</code>	<code>ccs_in_wait</code>
	NA	<code>ccs_in_pipe</code>
out	<code>mgc_out_stdreg</code>	<code>ccs_out</code>
	NA	<code>ccs_out_rdy</code>
	<code>mgc_out_stdreg_en</code>	<code>ccs_out_vld</code>
	<code>mgc_out_stdreg_wait</code>	<code>ccs_out_wait</code>
	<code>mgc_out_fifo_wait</code>	<code>ccs_out_pipe</code>
	<code>mgc_out_buf_wait</code>	<code>ccs_out_pipe (size=1)</code>

New commands for hierarchical blocks (10.5)

In 10.5, the `solution design set` and `solution design unset` commands were introduced to allow the user to add or remove certain types or flags to a hierarchical block. These commands were intended to replace the `DESIGN_HIERARCHY` directive and allow better control of hierarchical blocks in the design.

In 10.6, the `DESIGN_HIERARCHY` directive has been officially deprecated. The `DESIGN_HIERARCHY` directive is automatically mapped to the corresponding solution design commands for backwards compatibility.

Release 2025.2_1

This release provides bug fixes since the 2025.2 release. No options, directive or commands were added, removed or changed in this update release.

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-41264 Allow users to reverse `ac_window_v2_lflush` and `ac_window_v2_flush`'s padding format for even size kernels
- CAT-40976 channel `FIFO_DEPTH` is not propagated across interfaces in the design hierarchy
- CAT-40964 Allow type conversion on assignment when using `ac_array`
- CAT-40890 AI NN: Create Template Specialized `DenseLatency` function but with `II=RF` inline the current Unroll Factor = RF
- CAT-40872 Allow users to reverse `ac_window_v2_2d`'s padding format for even-sized kernels
- CAT-33367 Long runtime with combinational CCORE, seq CCORE synthesizes in one minute.

Release 2025.2

The following topics describe the changes since the 2025.1_1 release. The Changes to Directives/Options/Commands/Pragmas section describes all new, removed or changed items. The following section describes the enhancements that were added since the last quarterly release.

NOTE: *In previous years, Catapult had two major releases per year. Starting with 2025.1, Catapult will release four quarterly each year.*

Enhancements/Changes

The following sections describe the enhancements that were added in this release:

- Synchronization of Explicit Objects in SystemC (ac_sync)
- SystemC Support for Non-Blocking Peek
- Automatic Dual-Port/N-Port Wrapper Generation for SinglePort RAMs
- Passing ROM through Hierarchy
- HLS IP Enhancements
- Improvements for inferring Reset in SystemC Designs

Synchronization of Explicit Objects in SystemC (ac_sync)

In previous releases, Catapult supported the *SyncChannel* class from the *Connections* library to provide synchronization mechanisms between processes. Synchronization channels essentially define standalone handshaking signals, enabling users to directly manage synchronization. The *ac_sync* library provided additional synchronizing options but was previously only supported for C/C++ designs.

In this release, this functionality has been enhanced to enable the synchronization of explicit objects (eg. shared memories) in SystemC designs. This allows SystemC threads to synchronize at the object level in addition to global synchronization mechanisms. This enhancement results in improved Quality of Results (QoR) and concurrent execution of SystemC threads while coordinating access to shared data using a flexible synchronization mechanism provided by the *ac_sync* library.

- *Global synchronization* schedules SyncChannel objects with special rules where no other I/O can be scheduled on the same clock edge as a SyncChannel. This ensures *sync_in()* causes a block to wait until told to start, and *sync_out()* signals that a block is done and releases control to another block.

- *Per-object synchronization* provides precise control over shared resources by passing the object to be synchronized as a parameter to `sync_in()` and `sync_out()` calls. This approach enables specific scheduling of the shared resource access while maintaining concurrent execution, where possible. The feature is mainly useful for controlling shared memory access modeled in SystemC between SystemC threads with `II=1` pipelining.

The following example shows a simple design with two threads using per-object synchronization of a shared memory. Each thread independently coordinates memory access through `sync_in` and `sync_out` calls.

```
#include <mc_connections.h>
#include <ac_shared.h>

SC_MODULE(dut)
{
    sc_in<bool> CCS_INIT_S1(clk);
    sc_in<bool> CCS_INIT_S1(rst_bar);

    Connections::In<ac_int<32> > CCS_INIT_S1(in1);
    Connections::Out<ac_int<32> > CCS_INIT_S1(out1);

    SC_CTOR(dut) {
        SC_THREAD(thread1);
        sensitive << clk.pos();
        async_reset_signal_is(rst_bar, false);
        SC_THREAD(thread2);
        sensitive << clk.pos();
        async_reset_signal_is(rst_bar, false);
    }

    void thread1() {
        bool ping_pong = false;
        in1.Reset();
        sync1.reset_sync_out();
        wait();

#pragma hls_pipeline_init_interval 1
#pragma pipeline_stall_mode flush
        while (1) {
            for (int i=0; i < 16; i++) {
                mem[i + (16 * ping_pong)] = in1.Pop() + 0x1000;
            }
            sync1.sync_out(mem);
            ping_pong = !ping_pong;
        }
    }
}
```

```
void thread2() {
    bool ping_pong = false;
    out1.Reset();
    sync1.reset_sync_in();
    wait();

#pragma hls_pipeline_init_interval 1
#pragma pipeline_stall_mode flush
    while (1) {
        sync1.sync_in(mem);
        for (int i=16; i != 0 ; ) {
            out1.Push(mem[--i + (16 * ping_pong)] + 0x2000);
        }
        ping_pong = !ping_pong;
    }
}

private:
    Connections::SyncChannel CCS_INIT_S1(sync1); // memory sync between threads
    ac_shared<ac_int<32> [32]> mem;                // shared mem between threads
    bool d = ac::init_array<AC_VAL_0>(mem, 32);    // initialize memory to 0
};
```

In addition to SyncChannel, Catapult also supports SyncIn and SyncOut classes from the Connections library and p2p_sync<>::chan<> from the ccs_p2p library for per-object synchronization.

Following are the known limitations of this enhancement:

- In SystemC simulation, sync_in and sync_out calls halt the entire thread at wait() operations, while in hardware only the synchronized object operations are synchronized. This difference in execution ordering cannot be accurately represented in SystemC source simulation, making it impossible to precisely match the parallel behavior achieved in hardware.
- Not all object types can be passed to the sync_in/sync_out methods. For shared memories, the ac_shared object should follow the same coding guidelines as used for C/C++.

For an example of using this new functionality within Catapult, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Synchronization of Explicit Objects in SystemC**).

SystemC Support for Non-Blocking Peek

In the previous release, Catapult added support for blocking and non-blocking peek for C/C++ designs.

In this release, Catapult added support for non-blocking peek for SystemC designs.

For SystemC designs, the following new member functions are included in the *Connections* library. The functions do not consume any clock cycle and returns *true* if data is available in the channel otherwise returns *false*. Data available in the channel is passed using the function argument. The data can be inspected for

appropriate properties and a *Pop()* /*PopNB()* can be performed when appropriate conditions are met to read data from the channel.

```
bool Connections::In<>::PeekNB(T &data)
bool Connections::Combinational<>::PeekNB(T &data)
```

The following example shows how to use non-blocking peek (*.PeekNB()*) and pop (*.PopNB()*) in a SystemC design:

```
typedef ac_int<4, false> idata;
typedef ac_int<12, false> odata;
SC_MODULE(join_read)
{
public:
    sc_in<bool> CCS_INIT_S1(clk);
    sc_in<bool> CCS_INIT_S1(rst);
    Connections::In <idata> CCS_INIT_S1(in1);
    Connections::In <idata> CCS_INIT_S1(in2);
    Connections::In <idata> CCS_INIT_S1(in3);
    Connections::Out<odata> CCS_INIT_S1(out1);
    SC_CTOR(join_read) {
        SC_THREAD(main1);
        sensitive << clk.pos();
        reset_signal_is(rst, true);
    }
    void main1() {
        in1.Reset(); in2.Reset(); in3.Reset();
        out1.Reset();
        wait();
        #pragma pipeline_init_interval 1
        while (1) {
            idata dat1, dat2, dat3;
            if(in1.PeekNB(dat1) & in2.PeekNB(dat2) & in3.PeekNB(dat3)) {
                odata op = 0;
                in1.PopNB(dat1); in2.PopNB(dat2); in3.PopNB(dat3);
                op.set_slc(0, dat1); op.set_slc(4, dat2); op.set_slc(8, dat3);
                out1.Push(op);
            }
            wait();
        }
    }
};
```

Automatic Dual-Port/N-Port Wrapper Generation for SinglePort RAMs

In the previous release, when generating a memory library for Catapult, MemGen provided the option to automatically create a SystemVerilog wrapper that enabled single-port RAMs to act as simple Dual-port (1R1W) RAMs. If your technology only provided single-port RAMs, this functionality provided a method for using the single-port RAMs with Shared Memories in Catapult (which require 1R1W or dual-port memories).

In this release, MemGen extended the number of available Single-Port RAM wrappers to also include Dual-Port and N-Read/M-Write Port implementations.

Passing ROM through Hierarchy

If a constant array is defined in the top-level design or as a global constant array, and this array is accessed within a block through the block's interface, Catapult can map the constant array to a sequential ROM memory component.

```
#include "fir_filter.h"

#pragma hls_design block
void fir_filter (ac_channel<in_datatype > &input, in_datatype coeffs[NUM_TAPS],
ac_channel<in_datatype > &output )
{
    static in_datatype regs[NUM_TAPS];
    int temp = 0;
    int i;
    SHIFT:for ( i = NUM_TAPS-1; i>=0; i--) {
        if ( i == 0 )
            regs[i] = input.read();
        else
            regs[i] = regs[i-1];
    }
    MAC:for ( i = NUM_TAPS-1; i>=0; i--) {
        temp += coeffs[i]*regs[i];
    }
    output.write(temp>>7);
}

#pragma hls_design top
void myproject (ac_channel<in_datatype > &input, ac_channel<in_datatype > &out-
put ) {
    in_datatype coeffs[NUM_TAPS] = {123, 456, 789, 234, 567, 890, 345, 678,
                                     13, 56, 89, 24, 57, 18, 55, 78,
                                     15, 23, 24, 65, 110, 90, 37, 42};

    fir_filter(input,coeffs,output);
}
```

```
}
```

A constant array cannot be shared across multiple blocks. When a constant array is accessed by two different blocks through their interfaces, Catapult creates separate copies of the constant array for each block and connects each copy to its corresponding ROM port.

There are some limitations to the current flow:

- Only sequential ROMs are supported.
- Multi-port ROMs where the size of the constant array is not a power of two is not supported.
- Constant array cannot be shared across multiple blocks. If a constant array is accessed by two different blocks through their respective interfaces, Catapult generates separate copies of the constant array for each block and assigns each copy to its own dedicated ROM port.

For an example of using this new functionality within Catapult, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > ROM Through Hierarchy**).

HLS IP Enhancements

The following models were added in this release:

- 8 bit and 10 bit coding support was added for RGB2YCbCr model for both full and studio swing equations (BT.601 Enhancements). The model was also tested for HLS, HLV and coverage flows.
- High-Performance 2D DCT-II for VVC (H.266). A hardware-efficient 2D DCT-II is implemented for 4×4 to 64×64 TUs with minimal latencies (7–67 cycles) and streaming support for both fixed and variable TU sizes. Verified with full code coverage, the design achieves 909 MHz on a 3nm cell library, making it ideal for real-time VVC-based video applications.

Improvements for inferring Reset in SystemC Designs

In this release, the coding style for inferring resets in SystemC design was enhanced for consistency with the synthesizable subset rules. The following summarizes the rules for inferring resets in SystemC designs used by Catapult.

SC_THREAD/SC_CTHREAD:

Each SC_THREAD/SC_CTHREAD must have at least one reset. If reset does not exist, Catapult should report an error message.

- can have 0-1 `async_reset_signal_is()`
- can have 0-1 `reset_signal_is()`

SC_METHOD:

The reset coding guidelines for SC_METHODs only apply to sequential processes (sensitive to a clock). These guidelines do not affect combinational SC_METHODs. Catapult requires that sequential SC_METHODs must have at least 1 reset.

- can have 0-1 async resets.
- can have 0-1 sync resets.

If it is an async reset, then user must specify posedge or negedge in sensitivity list. This is how Catapult recognizes that it is an async reset. The posedge/negedge used in the sensitivity list must match the polarity of the if statement (Catapult must emit error if it does not).

Within body of SC_METHOD, the first if () always specifies a reset action. The rest of the body of the SC_METHOD simply specifies the datapath action.

- If the SC_METHOD **has an async reset**, then the first if() statement must specify the async reset action.
- If the SC_METHOD **does not have an async reset**, then the first if() statement must specify the sync reset action.
- If the SC_METHOD **has both async and sync resets**, then the sync reset behavior is specified in the datapath code and is not treated in any special way by Catapult - it is simply treated like a normal input signal.

Changes to Directives/Options/Commands/Pragmas

The following sections denote any changes to directives, options, commands, and pragmas in this release.

Directive Changes

The following list describes the changes to the Catapult directives:

- **ADVANCED_DPOPT** (*default changed*). Catapult introduced new datapath operator optimization in 2024.2. In this release, it is enabled by default.
- **ADVANCED_DPOPT_COMPLEXITY_EFFORT** (*new directive*). Some configurations of large, complex operators (eg. DIV/Write Slice) may have long runtimes during datapath operator optimizations. This directive allows you to inform Catapult to use area and delay estimates from the Catapult library, based on operator complexity, instead of running DPOPT. The bit width of the operands, constants, and power-of-2 bit width are examples of issues that affect the operator complexity. By default, this directive is set to *high*. The allowed values are *low*, *medium*, and *high*.
- **DISABLE_INPUT_AND_GATING** (*new directive*). This directive disables the AND gate which is introduced to not allow the data line to propagate when the 'valid' signal is not high in the wait controller for non-blocking pop (.PopNB()).

- **OPT_CONST_DIVS** (*value type changed*). The datatype of this directive changed from boolean to enum (yes, no, auto). Default value changed from 'true' to 'yes'. For backwards compatibility, if you set this directive to 'true' or 'false', Catapult will issue a warning that boolean values were deprecated and automatically set the directive to 'yes' (true) or 'no' (false).
- **OPT_CONST_MULTS** (*default value changed*). Default value changed from 'use_library' to 'auto'.
- **STRICT_CHANSYNC_IO_SCHEDULING** (*new directive*). This directive informs the scheduler to enforce strict SystemC scheduling constraints between Sync and Message-Passing Interface (Channel) operations. It is disabled by default. This directive only affects SystemC designs. For a description of these rules, refer to the reference page in the User Documentation for this directive.
- **STRICT_WAITSYNC_IO_SCHEDULING** (*new directive*). This directive informs the scheduler to enforce strict SystemC scheduling constraints between wait() operations and SignalIO, VariableIO, Sync, and Message-Passing Interface (Channel) operations. It is enabled by default. This directive only affects SystemC designs. For a description of these rules, refer to the reference page in the User Documentation for this directive.

Option Changes

The following list describes the changes to the Catapult options (Tools > Set Options...):

- **Constant Multipliers** (*ComponentLibs*) The default value of this option changed from 'use_library' to 'auto'.
- **No Boundary Optimization** (*Design Compiler flow*) This new option allows you to override the no_boundary_opt variable from the Catapult library when synthesizing the RTL netlist in Catapult using the Design Compiler flow. For backwards compatibility, the default value is 'use_library'. This option can be set to 'use_library', 'true', or 'false'.
- **Enable ac_fixed VRA** (*OSCI flow*). This new option enables Value Range Analysis for ac_fixed datatypes. For an example of using this functionality within Catapult, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Value Range Analysis with AC_FIXED**).
- **Enable ac_int VRA** (*OSCI flow*). This new option enables Value Range Analysis for ac_int datatypes. For an example of using this functionality within Catapult, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Value Range Analysis with AC_INT**).
- **Transform FF Enable to Feedback Mux for coverage** (*Output*). If enabled, this option informs Catapult to implement enable logic on registers using a feedback MUX. This transform may increase code coverage of the RTL netlist in some cases. By default, this option is *false*.
- **Rules to Ignore** (*SpyGlassLint flow*). This new option allows you to specify a space-separated list of rules from the current goal which SpyGlass will turn off during the running of the currently selected goal. By default, this option is an empty list ({}).

Pragma Changes

The following new pragma was added in this release:

- **hls_iterations <integer>** : This pragma allows users to specify the ITERATIONS directives value for do-while loops in their source files. Similar to the directive, the hls_iterations pragma only specifies the number of loop iterations for reporting by Catapult. The pragma (and the ITERATIONS directive) do not affect the hardware generated by Catapult.

Command Changes

none

Corrected Problems

The following sections denote the customer-reported bugs and enhancements that were addressed in this release.

Catapult

The following 50 customer-reported problems were fixed in this release of Catapult:

- CAT-40823 Name based parameter overriding required for 1R1W wrapper in Memory Generation
- CAT-40810 Runtime regression in "go architect" (mux_opt_selector, add cache for failing cases)
- CAT-40809 Runtime regression in "go architect" (mux_opt_selector, full feature)
- CAT-40707 Catapult crashes when unrolling specific loop
- CAT-40633 Issues with "ac_leading_ones" function
- CAT-40614 Error when connecting interface with type ccs_out_prereg_vld to downstream block with no FIFO in between
- CAT-40613 False path constraints are commented out in SDC
- CAT-40607 Long runtimes in "go instance" with Ultra - HLS is OK
- CAT-40602 Design fails with assertion failure 2025.1 onwards
- CAT-40528 Schedule failure when mixing write to sc_signal and connections
- CAT-40356 Issues with SC_METHOD according to coding style of nested if-else statement

- CAT-40313 Error when running Verify_rtl_sv_msim.mk
- CAT-40183 Propagation of generic parameter "log2_sz" in custom FIFO
- CAT-40165 Propagation of generic parameter "sz_width" appears dependent on use of mgc_ioport
- CAT-40149 Multi-Cycle path constraints for DC appear to not be consistent with each other
- CAT-40042 Advanced Datapath operator SpyGlass linting failed with W416 Error in shiftersActions
- CAT-39975 QWAVE instance name does not match with SCVerify testbench
- CAT-39952 Parallel CCORE synthesis with CCORE_DEBUG=true is slow; high memory overhead
- CAT-39768 ROM options are not available for top-down CCORE
- CAT-39277 Duplicated output registers
- CAT-38915 Simulation discrepancy between cycle and rtl netlists due to X propagation
- CAT-38675 Catapult produces different RTL when building multiple times with same code/directives
- CAT-38499 Enhancement to allow user to update additional fields in Library Builder
- CAT-38324 Different behavior (scheduling) according to the coding style for sc_in/out/signal
- CAT-38312 Scheduler makes read() operation before Pop() when clock changes
- CAT-38110 scheduling issues in SystemC flow
- CAT-37819 Prevent selection of pipe comp with external pinassocs in SystemC designs
- CAT-37762 Failing RTL regression
- CAT-36854 Catapult compile error with multi-clock SystemC design
- CAT-36686 Catapult generated code after power optimization has flops without reset
- CAT-36394 Documentation for CLUSTER_INLINE directive should update
- CAT-35499 dpd_param dynamic index issue
- CAT-35293 Generate a meta file with pipeline control information for debug/visibility into RTL
- CAT-35274 LINT/DC Warning from unmapped is_idle port in ccs_out_buf_wait_v[4|5] and ccs_ctrl_in_buf_wait_v4
- CAT-35143 nangate 45nm seems to be missing characterization for component widths

- CAT-30770 Add ac_shared.h to open-source
- CAT-30396 CCS_BLOCK and CCS_DESIGN usage for SysC and C++
- CAT-30211 Catapult reporting word_width conflict for top-level constant when direct input unused
- CAT-28084 Runtime issue at 'go dpfsm'
- CAT-27386 Broken cross-probing for top module IF
- CAT-26324 Excessively Long runtimes in "go instance" in Catapult Ultra
- CAT-23772 Area increase of ~50% with GATE_REGISTERS True (Ultra)
- CAT-12026 inout port incorrectly inferred in MIO loop init
- CAT-11925 LowPower run-time excessive vs HLS
- CAT-9661 Array declaration in top level causing Unsupported inout port error during assembly
- SLEC-33337 CCoverCheck not able to source ccover_check_pre_build.tcl correctly
- SLEC-33392 SlecBase license error for SALT_LICENSE_SERVER
- SLEC-33361 SAT should not be reported in case of rolled loops
- SLEC-32394 Wrong Report in CDesignChecker for OVL according to data-type in SystemC
- SLEC-33022 Wrong Report in CDesignChecker for ABR according to data-type in SystemC

Release 2025.1_1

This release provides bug fixes since the 2025.1 release. No options or commands were added, removed or changed in this update release.

Enhancements

The following enhancements were made in this release:

CLUSTER_INPUT_REGISTERS/ CLUSTER_OUTPUT_REGISTERS directives added

If the CLUSTER_TYPE directive is set to 'sequential' or 'both', this directive specifies whether Catapult should place registers on the inputs or outputs of the cluster. Catapult adds the registers inside the cluster component. When CLUSTER_TYPE is set to 'combinational', this directive has no effect. If CLUSTER_TYPE is set to 'both', then it is possible that the scheduler may select the combinational cluster for the final design.

This new functionality aligns clusters with other design block types.

- If you want to place registers **on a cluster**, use the new CLUSTER_INPUT_REGISTERS/ CLUSTER_OUTPUT_REGISTERS directives.
- If you want to registers **on a CCORE**, use the INPUT_REGISTERS/ OUTPUT_REGISTERS directives.
- If you want to place external registers **on a component**, use the REGISTER_INPUT/ REGISTER_OUTPUT directives.

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-40482 Transpose Implementation of FIR Filter Fails in Simulation, while systolic and tree Pass
- CAT-40281 Nesting scoped CCOREs produces simulation mismatch
- CAT-40170 License request failed, pid = catapultgui, mgls_errno = 515
- CAT-40159 Simulation Mismatch with masked memory for shared memory design
- CAT-40150 Licensing error at go analyze when moving to 2025.1

- CAT-40113 AI NN: Simulation deadlock when using back2back conv1 layers w/ sigmoid activation
- CAT-40067 Build Script for Importing Dual Clock FIFO RTL Needs Replacing
- CAT-40015 Potential simulation mismatch with SystemC crc generator
- CAT-39670 Combinational loop introduced in combinational CCORE with implicit INOUT port
- CAT-39407 Duplicated SV package name in generated RTL
- CAT-39232 Add an option of clock domain selection for multi-core architectures
- CAT-38576 Using array of pointers to SystemC ports can cause submodules to be optimized away
- CAT-28161 BDD-based runtime increase in regating during "go instance"
- CAT-26338 Long runtimes in "go allocate", possibly stuck
- CAT-24221 Excessive runtime in "go allocate" : coding transformation reduced runtime significantly
- CAT-13279 Catapult stuck at go extract due to GATE_REGISTER set to true

Release 2025.1

The following topics describe the changes since the 2024.2_2 release. The Changes to Directives/Options/Commands/Pragmas section describes all new, removed or changed Directives and Options. The following sections describe the enhancements that were added since the last release.

Enhancements/Changes

The following sections describe the enhancements that were added in this release:

- Scope-based CCOREs
- Advanced Datapath Optimization enhancements (Beta)
- Broadcast AC Channel to multiple receivers on same channel more efficiently
- New DataPath Operator Optimizer
- Memory/FIFO Generator Enhancements
- Catapult Library Changes
- HLS IP Enhancements
- Parallel CCOREs (alpha)
- Constraint Sweeping using YAML files (alpha)
- Cross-probe 'analyze' schematic in Design Analyzer (alpha)
- Source Code C-Step Explorer in Design Analyzer (alpha)

Scope-based CCOREs

In previous releases, Catapult allowed you to specify *functions* or *class-member functions* as CCOREs. If you wanted to 'carve out' part of an existing function into a CCORE, you needed to create a new function for that section of the code.

In this release, Catapult added support for creating CCOREs by creating a local scope (statements enclosed within curly braces) within an existing function or class member function. You would use the `hls_design` pragma to designate the local scope as a CCORE. This new scope-based CCORE capability makes it significantly easier to add CCOREs to existing designs without major restructuring of the code. (eg adding functions).

The following example shows a simple design with a labeled and unlabeled scoped region within an existing function:

```
#pragma hls_design top
void top_run( int& myIn, int& myIn2, int& sum1, int& max1)
{
    #pragma hls_design ccore
    MY_SUM:{
        sum1 = myIn + myIn2;
    }

    #pragma hls_design ccore
    {
        max1 = (myIn > myIn2) ? myIn : myIn2;
    }
}
```

For unlabeled scoped regions, the CCORE naming will be either:

- <Class Qualified Function Name> _scope_<Count> - If scope is part of a class member function
- <Function Name> _scope_<Count> - If scope is part of a function outside of class

For labeled scoped regions the CCORE naming will be either:

- <Class Qualified function Name> _<LabelName> - If scope is part of a class member function
- <Function Name> _<LabelName> - If scope is part of a function outside a class

For an example of using this new functionality within Catapult, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Scope-based CCOREs**).

Advanced Datapath Optimization enhancements (Beta)

In the 2024.2 release, Catapult introduced the Advanced Datapath optimization feature. This feature provides significantly more accurate area and delay estimates of certain datapath components than Catapult's default estimation. This enables better resource sharing and utilization which can potentially improve area and performance in the Catapult netlist. This flow also improves the correlation between the final area and delay from Catapult compared to downstream synthesis tools like Design Compiler.

In this release, Catapult increased the number of operators supported by this flow plus key improvements to the data processing flow that allows for better handling of adder trees, shifters, write slices, constant multipliers, and constant dividers. For example, with the ADVANCED_DPOPT flow, adder trees are treated as individual components and their resource sharing is optimized, rather than decomposing them into individual 2-input adders. This results in a more efficient implementation and better area and timing estimates. Similarly, constant multipliers and dividers are preserved and estimated as whole components, instead of being dissolved into complex logic gates

For an example of using this new functionality within Catapult Ultra, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Advanced Datapath Opt**).

Broadcast AC Channel to multiple receivers on same channel more efficiently

In previous releases, output channels (`ac_channel`) could only be implemented in RTL using separate data registers to drive the 'data' port for each channel. While this guarantees that the data signal is held stable when the channel is inactive, it can result in "extra" registers for cases when the same data is being transmitted to multiple channels (a broadcast). Per-channel registers may also be considered as "extra" storage in cases where only one out of multiple channels can be written in a single cycle (a demux operation).

In this release, Catapult adds the ability to send the same data to multiple output channels while sharing a single data register to drive the data for all channels using the new `ac_broadcast` functions. This allows for area savings compared to using normal channel writes, which will utilize separate data registers to drive each output channel.

NOTE: When using `ac_broadcast`, other portions of design code cannot write to any channel in the broadcast group. The only write to channels in the group must be done using a single call to a broadcast function.

The following code shows a simple design using one of the new `ac_broadcast` functions:

```
#include <ac_broadcast.h>
void dut(ac_channel<DT> in_ch,
        ac_channel<DT> &ch_a,
        ac_channel<DT> &ch_b,
        ac_channel<DT> &ch_c)
{
    DT outval = some_calc(in_ch.read());
    ac_broadcast_all(outval, ch_a, ch_b, ch_c);
}
```

Calling the broadcast function is blocking, which means that execution does not proceed until the writes to all output channels are complete. In hardware, channels in the broadcast group have independent flow control, which allows transfers to each channel endpoint to proceed as soon as possible. Once data is accepted for all channels (indicated by 'ready') the broadcast is complete.

Channels that are arguments to the broadcast function are placed in a "broadcast group" which introduces restrictions for the grouped channels. Other portions design code cannot write to a channel in the broadcast group. The only write to channels in the group must be a single call to a broadcast-family function.

For multiple examples of using this new functionality plus a complete description of the functions, best practices, and a list of restrictions, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Channel Broadcast and Demux**).

The following list provides a brief summary of the available functions in the `ac_broadcast` library:

- The `ac_broadcast_all()` function accepts a payload and multiple channels for which the payload will be written:

```
ac_broadcast_all(data, ch_a, ch_b, ch_c);
```

Additional variants of the `ac_broadcast()` function can alternately accept an array of channels, or a pointer to an array of channels:

```
ac_broadcast_all(data, ch_array);
```

- The `ac_broadcast_mask()` function is used for cases when it is useful to decide dynamically which output channels will be targeted, broadcasts can also be done using a write enable mask that specifies which output channels to target. The LSB of the mask corresponds to the first channel argument in the function.

```
ac_broadcast_mask(mask, data, ch_a, ch_b, ch_c)
```

The masked broadcast function also has alternate implementations that accept an array of channels, or a pointer to an array of channels:

```
ac_broadcast_mask(mask, data, ch_array);
```

- The `ac_demux()` function targets a single channel out of multiple in the demux group, based on a one-hot selector. A single data register will drive the data ports for channels in the demux group:

```
ac_demux(data, onehot_selector, ch_a, ch_b, ch_c);
```

An alternate implementation supports receiving a reference to an array of channels:

```
ac_demux(data, onehot_selector, out_chs);
```

The `ac_typed_demux()` function is a 'typed' variant of `ac_demux` that targets a single channel out of multiple channels that carry different payload types, and the data register is sized accordingly to fit any one of the channel payloads. The arguments are the selector and a set of DataPayload/Channel pairs:

```
ac_typed_demux(onehot_selector, data_a, ch_a, data_b, ch_b, data_c, ch_c);
```

Redundant RTL Code Elimination

In previous releases, the Catapult-generated netlist may include some RTL code that was optimized away by downstream RTL synthesis tools (eg. registers driven by constants). Although this RTL code did not affect the overall QofR, it may affect the RTL coverage during verification.

In this release, Catapult will remove redundant code from RTL netlist including constant registers and associated driver logic, which would otherwise be removed by downstream tools like Design Compiler. As a result, this new feature should reduce the number of RTL coverage issues while slightly improving the correlation between Catapult and RTL synthesis. Since the removed RTL was probably optimized by RTL synthesis, this feature should not affect the final QofR of the design.

You can control this new feature using the REDUNDANT_RTL_ELIMINATION directive. By default, this directive is enabled in Catapult Ultra.

```
directive set REDUNDANT_RTL_ELIMINATION true
```

NOTE: This feature is only available within Catapult Ultra. The directive is set to false and can not be changed in Catapult HLS.

For an example of using this new functionality within Catapult Ultra, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Redundant RTL Elimination**).

Memory/FIFO Generator Enhancements

In this release, the following changes were made to the Memory and FIFO Generators:

- SystemVerilog support for Memory and FIFO Generators
- 1R1W RAM Wrapper for Shared Memory Port in Memory Generator
- Memory Generator includes all memory types within a single library. Previously, MemGen created a library for each memory type.

Catapult Library Changes

In this release, the following changes were made to Catapult-supplied libraries:

- Library Builder can read gzipped liberty files
- Xilinx Versal-HBM (High-Bandwidth Memory) and Virtex UltraScale+ 58G libraries were added
- Dual-Port memories with port enables (DPRAMPE) were added to the Xilinx RAM library.

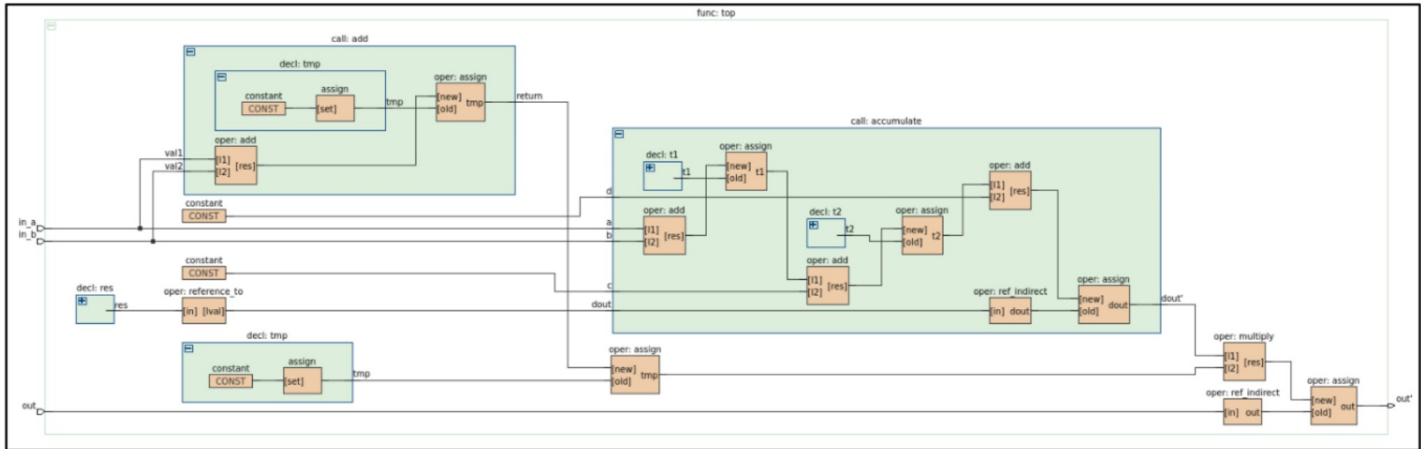
Cross-probe 'analyze' schematic in Design Analyzer (alpha)

In previous releases, Design Analyzer had minimal support for designs that did not pass the 'compile' stage. As a result, it was difficult to visualize the design's hierarchy and connectivity to locate the possible cause of the errors during the initial read of the design. In these cases, you often few debugging options other than scanning the source code or relying on text-based searching.

In this release, Design Analyzer can display most designs after the 'analyze' phase and allows you to cross-probe between schematic and source code views. This can be particularly useful for designs that are failing during synthesis, as the schematic view allows engineers to visualize the connectivity of the design. This new feature allows you to effectively gain a deeper insight into the structure and behavior of a design prior to the compile stage.

To use the schematic visualization and cross-probing features, your design must have the top-level defined using the *hls_design* pragma, and the design has progressed at least to the analyze stage in Catapult. The

key functionalities of this new feature include navigating the schematic view, making selections, using filtering and reader/writer analysis, and leveraging cross-probing between the schematic and other design views.

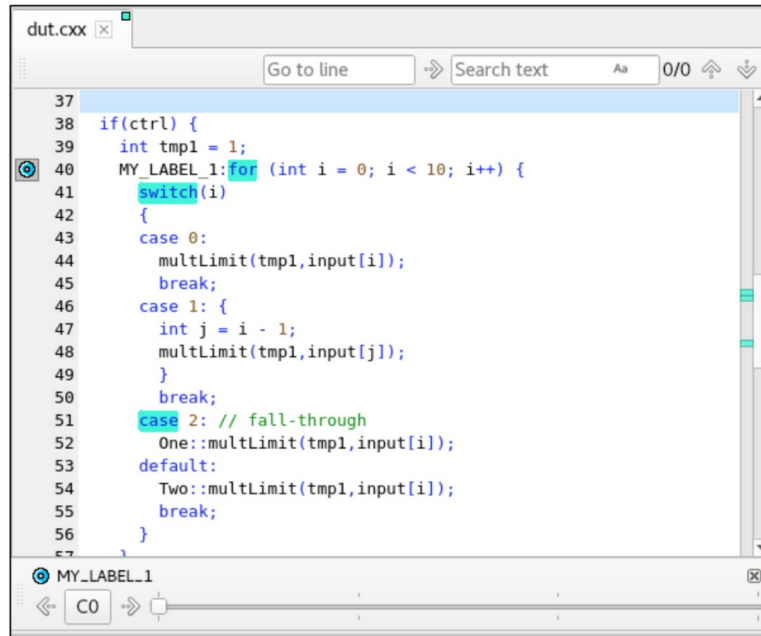


For step-by-step guidance on using this new functionality on some demo designs, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Cross-Probing an Analyze Schematic**).

Source Code C-Step Explorer in Design Analyzer (alpha)

For some designs, it is helpful to understand how the source code translates to the scheduling of IO and operations. This new alpha feature in Design Analyzer provides a visual representation of which areas of your source code translates to specific a CStep. For a given CStep, the tool will highlight all the pertinent code. This involves any operations, function calls, or variable read/writes that occur at the selected clock cycle in hardware. You can watch sections of your design highlight/unhighlight to better understand when source code operations take place, all in one condensed view.

The CStep tool is initially based on loops, and tracks the loops in the gantt chart in the Schedule view. The selectable loops are indicated in the margin of the Input Files. When selected, you can use the slider at the bottom of the Source Code window to navigate from CStep to CStep:



```

37
38 if(ctrl) {
39     int tmp1 = 1;
40     MY_LABEL_1: for (int i = 0; i < 10; i++) {
41         switch(i)
42         {
43             case 0:
44                 multLimit(tmp1,input[i]);
45                 break;
46             case 1: {
47                 int j = i - 1;
48                 multLimit(tmp1,input[j]);
49             }
50             break;
51             case 2: // fall-through
52                 One::multLimit(tmp1,input[i]);
53             default:
54                 Two::multLimit(tmp1,input[i]);
55                 break;
56         }
57     }

```

For an example of using this new functionality, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Source Viewer Cstep Explorer**).

HLS IP Enhancements

In this release, the following enhancements were made to HLS IP libraries:

- Added color space conversion toolkit example (ac_csc) which demonstrates the HLS, HLV and coverage flows.
- Added CCOV and CDesignChecker flows to the AC Window 2.0 Line Flushing (ac_window_v2_lflush) Interpolation toolkit.
- Added line repetition to AC Window 2.0 Flushing Support toolkit. This feature is used for algorithms like image interpolation.

Parallel CCOREs (alpha)

In previous releases, when Catapult identified a CCORE that was not in the existing cache, it generated any new CCOREs serially.

In this release, the parallel CCORE building feature addresses this challenge by allowing multiple CCORE blocks to be synthesized concurrently using separate worker processes. These processes are run on either the localhost or on the farm. Users can configure the number of parallel workers to use and each worker requires a separate Catapult license. This approach provides flexibility to tune the parallelism based on the specific design requirements and available compute resources.

By setting the `CCORE_WORKER` directive to a non-zero value (denoting the number of workers), Catapult will seamlessly integrate the parallel CCORE building functionality into the Catapult GUI. The progress of the parallel CCORE tasks can be tracked through transcript messages and output log files. This provides visibility into the build process and allows users to monitor the progress and identify any potential issues or bottlenecks.

```
directive set CCORE_WORKERS 2
```

The simple example design included in the toolkit demonstrates how to set up and run parallel CCORE building using Tcl scripts that can be customized for different numbers of CCOREs and parallel workers. Refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Parallel CCORE Building**).

Constraint Sweeping using YAML files (alpha)

Constraint sweeping typically involves running your design through Catapult while varying some constraints (eg. clock period) on each run. Users will analyze the results of the collective runs and select the solution that best meets their needs. In previous releases, this technique was accomplished by running scripts outside of Catapult.

In this release, you can automate the exploration of design alternatives by setting up a YAML file for your design and allowing Catapult to handle the sweeping of constraints. YAML is a human-readable markup language that provides a structured way to automate design flows and exploration. This allows you to efficiently explore a wide range of design configurations and parameters.

At the core of the constraint sweeping YAML flow is the ability to define variables that can be swept across multiple values. This allows you to automatically generate and evaluate numerous design alternatives, rather than having to manually set up and run each configuration individually. The YAML syntax uses "variable blocks" to define default values and "expand blocks" to specify the ranges of values to sweep. The individual design stages can then reference these variables, making the flow highly flexible and reusable.

To execute the constraint sweeping project, the YAML file is run from the Catapult GUI. Catapult will automatically build multiple solutions based on the variable sweeps, and it can also share work between similar configurations to improve efficiency. This new workflow allows you to rapidly explore the design space and identify the optimal set of parameters, without getting bogged down in repetitive manual tasks.

For an example of using this new functionality within Catapult, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > YAML Project**).

Changes to Directives/Options/Commands/Pragmas

The following sections denote any changes to directives, options, commands, and pragmas in this release.

Directive Changes

The following list describes the changes to the Catapult directives.

- **BOUNDARY_OPT** (*changed values*). This directive only affects optimizations across CCORE boundaries. In order to clarify the intent of this directive, the allowed values were changed from *boolean* to 'ccore'|*off*. For backwards compatibility, the default value is 'ccore'. Catapult will accept boolean values but issue a warning and transcribe the corresponding enumerated value.
- **DSP_FIR_ADD_INPUT_REGISTERS** (*new directive*) This directive informs Catapult to add the specified number of input registers to the data lines of transpose FIR filters.
- **EFFORT_LEVEL** (*new allowed value*). In addition to *medium* and *high*, this directive will also accept a value of *maximum*.
- **REDUNDANT_RTL_ELIMINATION** (*new directive*) This new directive improves RTL coverage and RTL synthesis correlation by removing redundant code from the RTL netlist including constant registers and associated driver logic, which would otherwise be removed by downstream tools like Design Compiler. For more information on this flow, refer to New DataPath Operator Optimizer.

Option Changes

The following list describes the changes to the Catapult options (Tools > Set Options...):

- **Location of Python Virtual Environment** (*HLS4ML flow*) The default value of this option changed from `$HOME/ccs_venv` to `$HLS4ML_VENV`. For backwards compatibility, if you do not change this option value and `$HLS4ML_VENV` is not set, Catapult will set this value to `$HOME/ccs_venv`.
- **Analytics file update period in minutes** (*Messages*) This new option specifies the approximate time interval for Catapult to display additional information on optimization analytics (only if `Messages/OptimizationAnalytics` is set to true). By default, this value is 30.
- **Netlist addsub** (*Output*): This option is a preliminary feature and should not be used. By default, it is set to *false* and the feature is disabled.
- **Netlist Banner** (*Output*) This new option provides control over the inclusion of timestamp and username sections of the comment banner at the top of the RTL netlist. Setting this option to 'none' restricts netlist comparisons in different solutions to only highlight RTL code changes. For backwards compatibility, the default value of this option is 'full'.
- **Path to Valgrind** (*Valgrind flow*): The default value for this option changed from `/usr/opt/bin/valgrind` to `/usr/bin/valgrind`.
- **use_dsp** (*Vivado*) By default, Catapult allows Vivado to determine whether to map pipeline multipliers to DSP blocks. If this option specifies 'yes' or 'no', then Catapult adds a `USE_DSP` property with this value to all pipelined multipliers, MAC operators, and complex multipliers in the Catapult-generated SDC file.

Command Changes

None

Pragma Changes

none

Corrected Problems

The following sections denote the customer-reported bugs and enhancements that were addressed in this release.

Catapult

The following 68 customer-reported problems were fixed in this release of Catapult:

- CAT-39961 Leaky_ReLU compile error on design
- CAT-39628 SCHED_USE_MULTICYCLE directive doc example is not consistent
- CAT-39567 DC warning on assertions in RTL
- CAT-39546 SSL issues when using create_env_sh
- CAT-39529 ac_broadcast: RTL is unexpectedly created with .sv extension
- CAT-39528 Fix function name typo in ac_broadcast.h
- CAT-39527 Add requested ac_demux alias functions to ac_broadcast.h
- CAT-39437 Multiple CCORE synthesis in parallel- Catapult Crashes
- CAT-39305 Missing Data in library characterization with DesignWare components
- CAT-39290 Designs crash at go compile in 2024.2_1, but work with 2022.2
- CAT-39208 REMOVE_UNUSED_DIRECT_INPUTS=true doesn't remove the port from SCVerify wrapper.
- CAT-39201 Document about Mapped BU flow
- CAT-38938 Enhancement request for adding each port enables in shipped DPRAM in ccs_xilinx
- CAT-38918 Catapult translating incorrect RTL with CCORE
- CAT-38907 Catapult synthesizing additional registers in 2024.2
- CAT-38875 Enhancement Request: Information linking SysC/C++ code to Generated RTL

- CAT-38832 ENABLE_DIRECT_INPUT_PUSHING ignoring the MAX_LATENCY
- CAT-38786 Memgen ignores user override of READ_DELAY if liberty file also used
- CAT-38783 ccs_in_wait_gated component not getting mapped when peek used on ac_channel arrays in loops
- CAT-38781 scverify reports mismatches on design
- CAT-38716 Error in propagation of ccs_in_wait_gated component in bup
- CAT-38692 BUP C++ design with ac_shared gives Internal Assertions in "go schedule"
- CAT-38681 Resource options not settable in constraint editor
- CAT-38677 unexpected duplication of ROMs
- CAT-38674 Catapult GUI options are frozen when selecting amba.ccs_axi4stream_full_in or amba.ccs_axi4stream_full_out
- CAT-38642 Support for Xilinx xcvu29p-fsga2577-2L-e
- CAT-38600 Enhancement for logically tied signal
- CAT-38598 Doc Update: IO_MODE default value is "super"
- CAT-38559 ADVANCED_DPOPT produces worse QoR sharing a right shifter
- CAT-38545 Maximum mergeable array size information
- CAT-38505 Improve deadcode cleaning in case of mixed Arithmetic/Logic operations
- CAT-38428 Optimization of register where data is either a constant or a ROM
- CAT-38383 ENABLE_DIRECT_INPUT_PUSHING affects the scheduling of normal sc_in
- CAT-38331 spyglass lint_rtl errors when the 'ADVANCED_DPOPT' directive is turned on
- CAT-38325 Incomplete Component Selection when choosing wr mask for mem (num byte enable)
- CAT-38244 Enhance Mktest to use absolute paths
- CAT-38233 SPRAM memory with Write Enable not scheduling for certain depth size
- CAT-38205 SCVerify sim mismatch when using SPRAM 1R1W wrapper
- CAT-38203 GUI crash at go extract
- CAT-38122 Request to have no change in output Verilog from the same SystemC or C++

- CAT-38102 VCS lacks comparison name under SCVERIFY
- CAT-38071 Block Floating Point accumulator
- CAT-37931 Selectively enabling VRA flow on variables in a design: Implementation.
- CAT-37903 catapult 24.1_2 still set interconnection to pipe unexpectedly
- CAT-37635 VRA post-processing issues with design
- CAT-37596 Inquiries and Enhancement Request for ac_probe split
- CAT-37417 RAMs with write-enables only works when RAM has 2048 addresses or less
- CAT-37347 could catapult show which port not match when doing CCORE mapping
- CAT-37223 Support nb_write_broadcast, write_demux - change request
- CAT-37121 Enhancement request for latency=5 option for Xilinx RAMs
- CAT-37099 FPGA tech. lib enhancement request
- CAT-36895 Single ac::wait() introduces 2 cycles between IOs
- CAT-36847 Clean Questa flows with Visualizer
- CAT-36638 X_PHD_SYNTHESIS doc example is functionally broken
- CAT-35950 Support parallel CCORE creation/characterization
- CAT-35947 Poor register QoR when updating slice of output port width
- CAT-35938 Catapult can't find the better resource sharing even though it exists
- CAT-35706 Loop Pipeline group children are sometimes disabled even when Pipeline is enabled
- CAT-35238 Support SV style RTL generation
- CAT-35107 Assertion failure when public member function marked as CCORE
- CAT-32768 Enable-signal selection disappears in GUI after setting it
- CAT-31561 Connections::Sync cannot assign same C-STEP and cause performance Problem
- CAT-30362 LB wizard should read compressed lib with gzip
- CAT-30121 ac_shared + {[Registers]} produce: Error Internal Bottom Up: word width conflict and resource size mismatch

- CAT-29421 Inform user when FSM_BINARY_ENCODING_THRESHOLD has been exceeded
- CAT-28212 allow ENABLE_RESET_TOGGLE to select async or sync reset
- CAT-27035 Questa CoverCheck not auto-waiving coverage for local functions of HLS RTL parameterized modules
- CAT-26517 GUI doesn't update BASE_BIT and BASE_ADDR for "mem2" when I choose a different "Packing Mode"
- CAT-25865 ram based pipe has 4 cycles of latency

CCoverCheck

- SLEC-32223: Enhanced coverage target support when CCOV keeps loop unrolled.
- SLEC-32811: Fixed additional license consumption issue with 2024.2 release.
- SLEC-32857: Simplified the command usage and its help.
- SLEC-32868 Fixed issues discovered in handling of arrays of ac_channel.
- SLEC-32910: Simplified user experience by hiding the deprecated JSON flow.
- SLEC-32718: Internal: Fixed reporting issue in printing the report in static flow.
- SLEC-32562: Enhanced support for supporting compact DU.
- SLEC-32639: Fixed issue related to coverage holes drop in macro expansion.
- SLEC-32611: Fixed ICC for pre/post increment operators.
- SLEC-31930: Fixed ICC related to compiler flag propagation issues.
- SLEC-31338: Fixed ICC related to initialization statement.
- SLEC-32756: SystemC: Corrected uniquifying cover_stmt based on method_register_stmt.
- SLEC-32299: SystemC: Fixed mismatch in column number for module instance declaration.

CDesignChecker

- SLEC-32942: Fixed issue related to pointer-to-pointer.
- SLEC-32964: Fixed misleading OVL CDesignChecker error messages on ac_fixed
- SLEC-32719: Fixed crash with ac_fixed for sub and div operators used together.

- SLEC-32535: Fixed UMR is not reported for array uninitialized with AC_VAL_DC.
- SLEC-32456: Support reset(), peek() and nb_peek() for ac_channel.
- SLEC-32178: Fixed issue in UMR reporting due to temporary creation.
- SLEC-32394: Fixed OVL reporting issue according to datatype in SystemC.
- SLEC-32816: Fixed incorrect message in CDesignChecker when violating OVL
- SLEC-33022: Fixed incorrect reporting in CDesignChecker for ABR according to data-type in SystemC
- SLEC-31130: Support async_reset_signal_is() with SC_METHOD call.
- SLEC-32577: Support to_double() API in ac_fixed.
- SLEC-32829: Support incremental runs which reuse previous solver results.
- SLEC-33015: Fixed ALS not reported in CDesignChecker

CFormalAssert

- SLEC-32482: Invalid mapping of probe signal in debug flow when struct member is an array.
- SLEC-31261: Time and value are missing for coverable CVR in results.log.
- SLEC-32733: SystemC and Matchlib Connections support in Debug Flow.

CFormalSLEC

- SLEC-32067: SC-403 Fail in cat2slec due to redeclaration of member function in spec_wrapper.h.
- SLEC-32679: Support ccs_ioport.{ccs_in_rdy_coupled and ccs_out_prereg_vld}.
- SLEC-32839: Support ccs_ioport.{ccs_out and ccs_ioport.ccs_out_rdy}.
- SLEC-32860: Support ccs_ioport.ccs_out_skidbuf_wait.
- SLEC-32871: Support ccs_ioport.ccs_in_pipe.
- SLEC-32888: Support ccs_ioport.ccs_in_wait_szchan.
- SLEC-32939: Support ccs_ioport.ccs_in_vld.
- SLEC-32358: Support class based hierarchical designs with private members.

Release 2024.2_2

This release provides bug fixes since the 2024.2_1 release. No directives, options or commands were added, removed or changed in this update release.

License Server Update 11.19.01 to 11.19.4.1

In this release, **it is highly recommended** that you use *saltd* version 11.19.4.1 (or later). Although Catapult only requires *saltd* 11.19.0.1 (or later), the newer version of *saltd* provides some license consumption bug fixes and runs natively on Rocky 9.X. This updated *saltd* license daemon provides immediate benefits plus prepares you for future OS updates.

As shown in the following example, you can determine the version of the current *saltd* daemon using the linux *lmstat* command. If you do not include the *-c* switch, *lmstat* will return the following information for all available license servers.

```
> # lmstat -c <port>@<host>
> $MGC_HOME/bin/lmstat -c 1717@my_lic_server

lmstat - Copyright (c) 1989-2022 Flexera. All Rights Reserved.
Flexible License Manager status on Thu 1/16/2025 13:32

License server status: 1717@my_lic_server
    License file(s) on my_lic_server: /opt/Siemens/ActiveLicenses/cat_hls.lic:

my_lic_server: license server UP (MASTER) v11.19.5

Vendor daemon status (on my_lic_server):

    saltd: UP v11.19.5
    mgcld: UP v11.19.5
```

If you are not currently using the expected *saltd* version, the recommended licensing software can be found at either of the following locations:

- You can download the *Siemens License Server* v4.2 (or later) package, which includes *saltd* v11.19.4.1. The *Siemens License Server* can be downloaded from Siemens SupportCenter.
- The Catapult install tree includes the FlexNet 11.19.5 package (which includes v11.19.5 version of the *saltd* daemon).

For more detailed instructions on using the *Siemens License Server*, refer to `$MGC_HOME/shared/pdffdocs/sw_siemens_license_server_install.pdf`.

Corrected Problems

The following 15 customer reported problems were fixed in this release:

- CAT-39201 Document about Mapped BU flow
- CAT-38822 Installed Example for Importing DUAL CLOCK FIFO RTL not working as expected
- CAT-38803 Report the unused interface resource blocks/port
- CAT-38788 Transaction done is not cleaned up in BUP for split memories with unused segments
- CAT-38745 High Runtime for Design with unused direct input in clustering
- CAT-38724 Catapult Ultra: using 2 licenses for each invocation.
- CAT-38714 Common License Code consuming multiple licenses
- CAT-38637 sif assert at go dpfsm due to WORD_WIDTH constraint on ac_shared variable in BUP
- CAT-38571 Xilinx memory enhancement request for read enable (re)
- CAT-37143 Compile error: Unsupported language feature 'template_param' (CIN-15)
- CAT-36881 sif_assert and no Clear error when 'void *' is present in the code
- CAT-36812 Simulation mismatch with inline variable on class data members
- CAT-36796 Segmentation fault on 2024.1_1 at "go compile" with C++17 feature- Inline variable
- CAT-36304 Customer Using C++20: Assertion during Compilation Stage
- CAT-29129 For ac black box Concat flow needs to support system verilog files

Release 2024.2_1

This release provides bug fixes since the 2024.2 release. No directives, options or commands were added, removed or changed in this update release.

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-38744 SIF ASSERT for bottom-up example with memories
- CAT-38706 Function Argument Type Mismatch in `resize_nearest` call
- CAT-38542 Catapult Crash at allocate
- CAT-38539 Memgen using DFT pins timing value for `read_ram` timing
- CAT-38536 When using `DIRECT_PORT` if an Out port isn't Pushed, `scverify` generates lots of 'x' conversion warnings
- CAT-38473 RAM write address is "X" when no pipeline constraint
- CAT-38429 Memory Generator cannot import header files included in SV memory model
- CAT-38352 Running Scverify when the architecture is more strict results in mismatches
- CAT-38349 Catapult Crash at allocate | Schedule , `ADVANCED_DPOPT = true`
- CAT-38348 pragmas not honored within `conrstexpr` if
- CAT-38299 Quartus running errors out on `-early_place`
- CAT-38271 naming issue of `ac_bank`
- CAT-38248 Connections PopNB interface samples input data without valid guard
- CAT-38184 New Connections `vld/rdy/dat` naming scheme breaks Connections back-annotation with `input.json`
- CAT-38139 Catapult 2024.1_1 - BitUnion2 in Connections compile error
- CAT-38136 Connection FIFO of size 1 does not achieve full throughput

- CAT-38107 output/CaseMux true creates incorrect RTL in mux's pessimism reduction
- CAT-37985 Internal assertion when #line control pre-processor directive used with invalid file path
- CAT-37410 dataflow `sif_assert` (`sif_df_vstat.cxx` line 219 and 223)
- CAT-36718 `sccom-6145` error in SCVerify for class based hierarchy design
- CAT-36588 Optimizing `float*fixed = fixed` – update documentation
- CAT-36460 Simpler style of naming banked arrays
- CAT-36179 2024.1-Design stuck running in go allocate
- CAT-35056 `sif_flows_api.cxx` line 4829: Bottom-up synthesis fails when leaf blocks has `ac_window` line buffers
- CAT-33580 Lib Wizard defaults vs Recommended settings in Docs
- CAT-24868 MemGen HDL parsing fails when FPGA library components in use
- CAT-9497 memgen: Verilog reader cannot handle port width expression in terms of `localparam`
- SLEC-32913 Issue with if else if hierarchy identifying unreachable as reachable (CCoverCheck)
- SLEC-32869 CPT-ABC: CCoverCheck crash due to multiple reasons
- SLEC-32868 CPT-ABC : CCoverCheck crash
- SLEC-32857 CoverCheck usage improvements
- SLEC-32811 CCoverCheck licensing issue
- SLEC-32719 CDesignChecker Crash with `ac_fixed` for sub and div operators used together
- SLEC-32621 CBU in CDesignChecker is not detected due to "`std::cout`"
- SLEC-32577 CDesignChecker CPT-NASE: conversion from `ac_fixed` to double
- SLEC-32535 UMR is not reported for array uninitialized with `AC_VAL_DC`
- SLEC-32456 Support for `reset()`, `peek()` and `nb_peek()` function in CDesignChecker
- SLEC-32403 Scope-based ccore message in CDesignChecker, not reported in logs for sysC design
- SLEC-32178 CDesignChecker not flagging UMR

Release 2024.2

The following topics describe the changes since the 2023.2_2 release. The Changes to Directives/Options/Commands/Pragmas section describes all new, removed or changed Directives and Options. The following sections describe the enhancements that were added since the last release.

Enhancements/Changes

The following sections describe the enhancements that were added in this release:

- New DataPath Operator Optimizer
- New Branch View in Design Analyzer
- IDLE signal logic generation for SystemC designs
- Value Range analysis with ac_int
- Mutually Exclusive sharing for multi-port memories
- Non-Blocking Peek (Beta)
- Automatic Splitting of Struct/Array datatype Interface ports
- Automatic smoothing of ASIC Library Data
- Added support for the AprisaRTL flow (Beta)
- Catapult Library Changes
- Added RTL Manifest flow

New DataPath Operator Optimizer

Although Catapult can estimate the area and delay of operations that can be implemented as a single module fairly accurately, the estimation for a group of operations, such as datapath arithmetic operations, can be more pessimistic. For instance, in the case of an adder tree (2-input adders connected in a tree-like structure), Catapult assumes that each adder calculates the sum until the end using some architecture (e.g. Ripple Carry Adder) and would claim that the resulting area is equal to the sum of each individual 2-input adder. In reality, a synthesis tool would realize that only the final sum is needed and would utilize a more efficient strategy, such as using Carry Save Adders for the intermediate adders, and only one Ripple Carry Adder for the output of the tree. This leads to Catapult having a pessimistic estimation in such cases. Catapult may also be more pessimistic in arithmetic operations involving constants or partial constants.

Such Datapath components will be estimated more accurately with the new Advanced Datapath Opt flow, using an efficient synthesis tool. This happens during the go instance stage, allowing us to use the improved estimation both for better sharing, and for more accurate final QoR reporting. The components affected by this flow are:

- *Adder Trees*
- *Shifters*
- *Write Slice*

To enable this flow, you need to set the following two directives:

- ADVANCED_DPOPT should be set to true. By default, it is false.

```
directive set /ADVANCED_DPOPT true
```

- ADVANCE_DPOPT_READ_LIB should be set to the full absolute path to the technology Liberty file.

```
directive set /ADVANCED_DPOPT_READ_LIB {full_path_to_library}
```

For an example of using this new functionality within Catapult Ultra, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Advanced Datapath Opt**).

Automatic Splitting of Struct/Array datatype Interface ports

In previous releases, Catapult supported structs on the interface and provided the ability to generate a RTL netlist with port splitting or structs retained on the interface as VHDL records and SystemVerilog structs using the PRESERVE_STRUCT and STRUCT_LAYOUT directives. When PRESERVE_STRUCTS=false, the structs were split during compile.

In this release, Catapult provides the STRUCT_SPLIT_MODE directive so the struct can be retain through the schedule stage and provide split ports in the final netlist without a wrapper layer. If the PRESERVE_STRUCT directive is set to true(default) and the STRUCT_LAYOUT directive is set to c-style, the STRUCT_SPLIT_MODE directive provides control over splitting the interface ports at all levels of hierarchy at the end of the schedule stage, except ModularIO hierarchies (implemented as TD CCOREs). The split ports will be named based on the struct structure. This directive is applicable to the interface ports of struct datatypes, including channel interface ports or sc_in/sc_out. In addition, interface ports that are of array datatype and not mapped to memories are also split based on the STRUCT_SPLIT_MODE. PRESERVE_STRUCTS directive is not applicable to non-channel interface ports and so, the struct datatypes of such ports are automatically split at compile stage into leaf level fields. Interface ports related to a memory will not be processed.

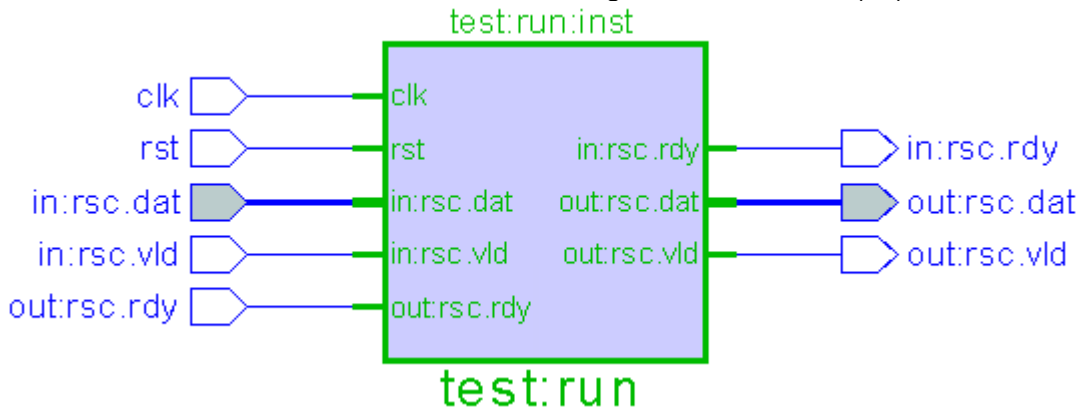
The applicable interface port datatype is an array (eg. array of struct) or a struct (datatype is struct containing fields that can be structs and/or arrays). This directive can be assigned one of the following values.

- **full** : Structs and arrays are split into elementary field elements.

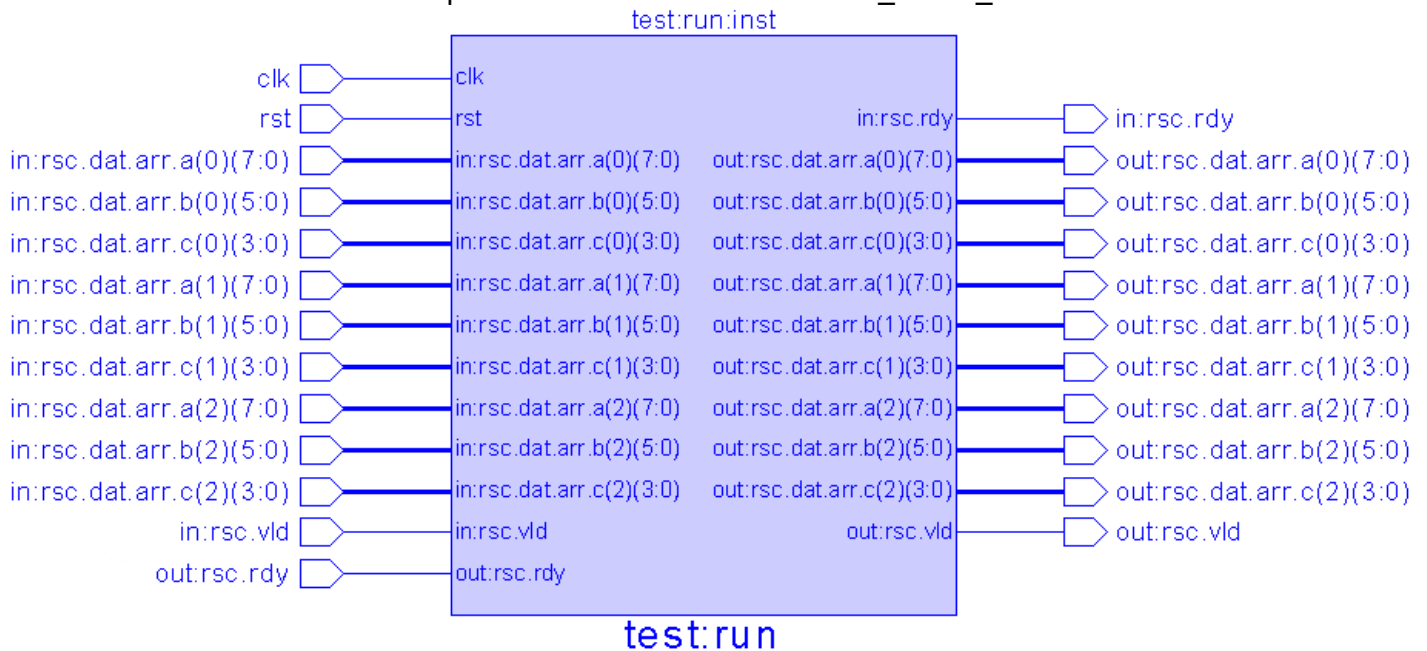
- **outermost** : only the outermost struct/array is split, every field of the outermost struct will retain its datatype and will be netlisted appropriately
- **off** (default) : No split is performed

For Bottom-Up blocks, the interface must be consistent with the instantiating block. Any mismatch due to an inconsistent setting of this directive will result in an error.

As an example, the first schematic shows the results of using the default value (off).



The second schematic show the top-level interface will the STRUCT_SPLIT_MODE directive set to "full".



For backwards compatibility, using default values for all directives (STRUCT_LAYOUT=legacy), Catapult still flattens structs during compile and writes them as a single vector in the RTL netlist.

Non-Blocking Peek (Beta)

In previous releases, `ac_channels` did not directly provide peek functionality. Although you could model 'peek' behavior using channel reads, the data had to be stored locally in a register (due to the destructive nature of channel reads) and the resulting implementation had additional latency with higher area and power costs.

In this release, Catapult provides initial support for the non-blocking peek functionality with `ac_channels` with the `.nb_peek()` method. This new enhancement provides users with the ability to "look-ahead" to the next available data in the channel, without issuing a read operation, and decide whether the design should read the data. If the data should be read, the resulting design will use a combinational logic path to raise the 'rdy' signal and issue a read operation within the same clock cycle as the non-blocking peek. With the proper coding style, the peek+read operations can be performed without any additional clock cycle delay (eg. no additional registers in the path).

```
bool nb_peek(T &val)
```

A `nb_peek()` member function, must return a true if channel valid is high. The data is returned in "val". If channel valid is low the function must return false. The `nb_peek` function should be applicable to `ac_channels`.

When the channel is empty, channel valid (vld) should remain low. A peek should check if channel valid is high and then capture the data value into a variable in C++.

In the Catapult netlist, this port is mapped to an interface resource containing a `ccs_in_wait_gated` component. This new component is identical to the default `ccs_in_wait` component except it adds 3 ports to track the peek/read operation. This new component does not support auto stall control.

```
1
2 #include <ac_fixed.h>
3 #include <ac_channel.h>
4
5 #pragma hls_design top
6 void nb_peek_ex (
7     ac_channel<int> ch[2],
8     ac_channel<int> &ch_out
9 )
10 {
11     int peek0 ;
12     int peek1 ;
13     bool good0 ;
14     bool good1 ;
15
16     good0 = ch[0].nb_peek(peek0) ;
17     good1 = ch[1].nb_peek(peek1) ;
18
19     int payload0 ; // payload to write out
20     int payload1 ; // payload to write out
21     int payload_out = 0 ; // payload to write out
22
23     bool write_chan = false ;
```

```

24
25 // Only one IF statement between the nb_peek and nb_read
26 // channel read of ch[0]
27 if ((good0 && good1 && (peek0 > peek1)) || (good0 && !good1)) {
28     ch[0].nb_read(payload0) ;
29     payload_out = payload0 ;
30     write_chan = true ;
31 }
32
33 // channel read of ch[1]
34 if ((good0 && good1 && !(peek0 <= peek1)) || (!good0 && good1)) {
35     ch[1].nb_read(payload1) ;
36     payload_out = payload1 ;
37     write_chan = true ;
38 }
39
40 if (write_chan) {
41     ch_out.write(payload_out) ;
42 }
43 }
PATH: $MGC_HOME/shared/examples/docs/modeling/hier/peek/nb_peek.cpp

```

This example is expected to pipeline at II = 1 and eliminate any storage registers that currently would be required on unused read values from ch[1] or ch[2].

Known limitations

- Each read()/nb_read() must be inside a single if-statement (it can be in the *else* too), but it cannot be in a nested if-statement. An if-then-else statement counts as nested if-statement. There can be more operations inside the *if* statement. All the other operations do not have to follow these restrictions.

```

channel.nb_peek(payload)
...
if (some complex condition) {
    channel.nb_read(payload);
}

```

- To avoid an extra register, do not peek values into a static variable.
- SystemC and MatchLib connections (using PeekNB() method) are not supported in the initial release.
- In this release, Catapult supports blocking and non-blocking reads (read()/nb_read()) with a non-blocking peek (*nb_peek()*) but blocking peek (*peek()*) is not supported.

IDLE signal logic generation for SystemC designs

In previous releases, Catapult only supported idle signal generation for C/C++ designs using the IDLE_SIGNAL directive.

In this release, Catapult added the *hls_idle* pragma which can be added to any signal bit logic port on a module interface.

For an example of using this new functionality using designs with multiple SC_THREADS or hierarchical designs, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > System Idle Generation**).

New Branch View in Design Analyzer

Debugging designs can be difficult and sometimes scheduling can fail due to either a feedback dependency or a resource competition failure. The Branch Viewer is a table that is filtered to show the control structures of a design to help understand how IO, loops, memories, and other resources are scheduled and optimized. The aim is to help mitigate problems associated with failures by providing a way to select allocate state branch objects and their children, filter to failures across all windows, cross probe, and view pertinent data.

The table data includes object name, type, start Cstep, start time (ns), delay (ns), total operations, info such as read / write port count, and failure type if applicable. The total operations column is a count of all operations under a given object. Loops are denoted with the loop icon, blocks are bolded, and initiation intervals and unroll count tags are prepended to object names. Resource competition reader rows have a red background, and the resources have teal backgrounds.

Two important buttons have been added to the toolbar: the branch selection button and the filter-to-failure button. The branch selection feature allows for the selection of branch control structures (loops, if, else, etc.) and all operations within the scope the of the branch, not including nested control structures and their operations. This selection will propagate to all other windows. The filter-to-failure button will filter the table to show only the failure operations and the operations leading to them. This filter will apply to all other windows as well. The filter can be removed by toggling the button again.

For an example of using this new functionality within Catapult's Design Analyzer, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Branch View**).

Value Range analysis with ac_int

In this release, Catapult added support to perform value range analysis (VRA) of *ac_int* variables in a C++ design. The core of this functionality is a runtime instrumentation feature added to the *ac_int* datatype that is available to Catapult users. This instrumentation captures the quantization and overflow errors occurring in an *ac_int* variable as it is being exercised by a C++ testbench. At the end of the program execution, a summary report is printed (either to the standard output or a file) for the *ac_int* variables used in the design, which represents the unprocessed VRA output. This report can be further refined using the postprocessing provided by Design Analyzer (DA), if it is printed to file. The final report should show the value ranges, RMS quantization errors, call stack to the location of each variable's declaration, and a suggested refined bit-width based on the values observed.

Enabling the value range analysis feature of the *ac_int* class requires the following steps:

- You must have a valid Catapult installation – the value range analysis instrumentation is not provided in the open-source AC Datatypes release.
- You must compile your program with debug symbols – i.e. use `-g`

- You must include the compiler flag `-DAC_INT_VRA`
- You must include the link library 'bfd' (the Linux Binary File Descriptor) when linking your program.

The following commands show a simple example of compiling and executing a program, with the key elements shown in bold:

```
% setenv LD_LIBRARY_PATH $MGC_HOME/lib:$MGC_HOME/shared/lib
% $MGC_HOME/bin/g++ -g -DAC_INT_VRA -I$MGC_HOME/shared/include \
$MGC_HOME/shared/examples/methodology/vra_ac_int/simple.cpp -lbfd -o simple
```

For an example of using this new functionality within Catapult's Design Analyzer, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Value Range analysis with ac_int**).

Mutually Exclusive sharing for multi-port memories

In previous releases, HLS allowed for the users to express mutual exclusivity of operations in their designs through the syntactic "if-else" conditions.

```
if (!sel1 & !sel2 & sel)
out = mem[0];
if (!sel & sel1)
mem[1] = addr1;
if(sel & sel2 & !sel1)
mem[5] = addr;
if (sel2 & !sel)
out = mem[6];
```

This enhancement allows the sharing of mutually exclusive operations inside IF-conditions that are not syntactic disjoint. This enhancement enables the user to schedule designs utilizing multi- port memories with multiple IF-conditions that are mutually exclusive without the need for writing syntactic if-else statements. It also enables the user to benefit in terms of overall reduced latency by sharing the mutually exclusive memory operations to the ports of multi-port memories.

With this enhancement, users can express the same by writing mutually exclusive conditions in multiple IF statements. For example, in the text box below, the conditions in the two if-statements are mutually exclusive, i.e., only one of the two if-conditions will always be executed in the testcase. This means that out of the two operations, only one will be used in the resulting synthesized design.

The user can schedule the above design using the following constraints:

```
solution library add ram_nangate-45nm-dualport-beh
directive set /top/core/main -PIPELINE_INIT_INTERVAL 1
set env(MULTIPORT_SCHED) 1
```

For an example of using this new functionality within Catapult's Design Analyzer, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Mutually Exclusive sharing for multi-port memories**).

Automatic smoothing of ASIC Library Data

During library characterization, Library Builder performs multiple RTL synthesis runs to generate Area/Delay data for parameterized library modules (QMODs) over a range of clock frequencies (datasets). Using this data, Library Builder generates Area and Delay interpolation equations for each library module (MOD) in the library. Catapult uses these equations to make tradeoffs during scheduling. Ideally, the points in the dataset should monotonically change within the dataset. In other words, as the clock period decreases, you would expect the RTL synthesis tool to generate faster implementations. If the RTL synthesis tool returns a non-monotonic point in the dataset, this inconsistency can skew the interpolation equation and cause non-optimal results from the scheduler.

In this release, Library Builder can identify and modify any QMOD dataset that contains non-monotonic changes. Library Builder will always retain the original characterization data so the RTL synthesis data is never lost. Library Builder performs this smoothing step whenever a library, QMOD, or datapoint has been characterized, re-characterized or if the `adjust_char_library.tcl` is executed. This change allows the scheduler to make better decisions and increases the correlation between Catapult and your RTL synthesis tool. If Catapult loads an existing Catapult library that has not been monotonized, a warning will be reported.

This feature is controlled by the following option within Library Builder (by default, it is set to *true*):

```
options set ComponentLibs/SmoothLibData true
```

NOTE: This feature only applies to Catapult libraries for ASIC technologies.

This new functionality provides the following additional features:

- Ability to export the characterization data (original or monotonized) in CSV format to evaluation

```
LB> library check -dump2csv 1
```

- Ability to revert monotonized library data with the original characterization estimates

```
LB> library check -revert 1
```

To monotonize the library data for an existing Catapult ASIC library, follow these steps:

4. Invoke Library Builder

```
catapult -product lb
```

5. Open the Catapult ASIC library (**File > Open Library...**)
6. Check and address any QMOD monotonic issues in the QMOD's dataset by updating the library

```
LB> library check -update 1
```

7. Save the library (**File > Save Library**)

Added support for the AprisaRTL flow (Beta)

In this release, Catapult and Library Builder added support for the AprisaRTL synthesis tool. This new functionality allows you to run placement-aware synthesis and ASIC place-and-route on your Catapult generated RTL netlist. Additionally, this flow supports the clustering and CCOREs flows within Catapult.

To use this flow within Catapult, your Catapult library must include AprisaRTL as an available flow. For example:

```
solution get /TECHLIBS/nangate-45nm_beh/FLOWS/* -match glob -return leaf
# DesignCompiler OasysRTL RTLCompiler AprisaRTL
```

In addition to synthesizing the RTL netlist generated by Catapult, you can also characterize an ASIC library using AprisaRTL in Library Builder. In this initial release, Multi-Voltage Threshold (MVT) and OnTheFly library characterization are not supported.

NOTE: If you use the Library Wizard to create a Catapult Library and want to use APRTL for characterization, after the wizard completes, you will need to manually change the `interconnect_mode` variable from "none" to "wireload" prior to characterizing any dataset.

For APRTL library characterization, the `fractional_digits_t` variable in the Catapult library must be set to a value less than 7. This variable specifies the number of digits used when gathering timing data for characterization. By default, this variable is set to 7 in this release.

Catapult Library Changes

In this release, the following changes were made to Catapult-supplied libraries:

- The Lattice Avant-E (LAV-AT) technology with speed grades 1,2, and 3) was added. This library supports the Radiant RTL synthesis tool.
- The `ccs_in_wait_gated` component was added to the default `ccs_ioport` library which contains interface components used in the top-level blocks. This component is only used the `nb_peek()` functions on `ac_channel` interfaces.
- Memory Generator added support for generating memory-based pipes with externalized RAM. You can inform Memory Generator to create this model in the generated library by using the checkbox in the last pane of the Memory Gennerator wizard or adding the `GEN_EXT_RAM_PIPE` variable to your existing MemGen Tcl script.
- The `ccs_pipe_pointer` component added user-settable parameters called `rdy_reg` and `vld_reg`.

Added RTL Manifest flow

In previous releases, after the extract stage, Catapult generated a file containing the top-level RTL netlist and each dependency file on a separate lines. Catapult generates this filelist for each HDL language (eg. VHDL, Verilog).

In this release, Catapult provides the RTL Manifest flow which allows you to generate one or more manifest files that can be used for each downstream tool flow. A separate manifest will be generated for each netlist type (VHDL and Verilog). Similar to the existing filelist, the manifest file lists the HDL files in bottom-up order.

The following options control the leaf names of the VHDL and Verilog manifest files (default values are shown). If the flow is enabled, Catapult will generate these files in the Solution Directory along with the top-level RTL netlist.

```
options set Flows/RTLManifest/VERILOG_MANIFEST rtl_manifest.v.f
options set Flows/RTLManifest/VHDL_MANIFEST rtl_manifest.vhdl.f
```

The Manifest flow provides the following value substitution strings to assist in the generation of meaningful names. You can utilize any of these substitutions in the *_MANIFEST option values:

- **%DU%** will be substituted with the top design's netlist name
- **%ARCH%** will be substituted with the solution VHDL architecture name (which is the solutions version number)
- **%BASENAME%** will be substituted with the basename of the RTL netlist file

For each dependency netlist file written to the manifest file, the VERILOG_FORMAT and VHDL_FORMAT options controls how tCatapult will generate the formatted string in the manifest file. The following examples show the usage of these options using their default values:

```
options set Flows/RTLManifest/VHDL_FORMAT {-work %LIB% %FILE%}
options set Flows/RTLManifest/VERILOG_FORMAT %FILE%
```

The Manifest flow provides the following value substitution strings to provide custom formatting for a particular downstream tool for each netlist file in the dependency list. You can provide these substitutions in the *_FORMAT option values:

- **%FILE%** will be substituted with the filename of the netlist dependency
- **%LIB%** will be substituted with the logical library name into which the file should be compiled
- **%DU%** will be substituted with the top design unit in the file (may be empty)
- **%ARCH%** will be substituted with the VHDL architecture name of the top design unit (VHDL only)

The flow will also have an option to specify the comment string prefix for any comments written to the manifest file.

```
options set Flows/RTLManifest/COMMENT {#}
```

No manifest file will be generated for the "Concat" netlists. Netlist dependencies that come from the Catapult installation will use a variable to specify the location of the Catapult installation.

If this flow is enabled, Catapult will automatically generate the manifest at the end of the 'extract' and 'power' stages. You can also explicitly generate the manifest file during these two stages by running the following Tcl command:

```
flow run /RTLManifest
```

Changes to Directives/Options/Commands

Directive Changes

The following list describes the changes to the Catapult directives.

- **ADVANCED_DPOPT** (*new directive*) This new directive enables the Datapath Operator optimizer. By default, it is disabled. For more information on this flow, refer to New DataPath Operator Optimizer.
- **ADVANCED_DPOPT_READ_LIB** (*new directive*) This new directive specifies the name of the technology Liberty (.lib) file. By default, the directive value is empty. For more information on this flow, refer to New DataPath Operator Optimizer.
- **ADVANCED_DPOPT_WLM** (*new directive*) This new directive specifies the wireload model from the liberty file that Catapult uses to estimate net capacitance and delay. By default, the directive value is empty. For more information on this flow, refer to New DataPath Operator Optimizer.
- **DSP_MAPPING** (*new allowed value*) This directive added *transpose* as an accepted value. The default value remains as *tree*.
- **ESTIMATE_LOGIC_DELAYS** (*CCORE inherit*). This directive is now inherited by Top-Down CCOREs and used when generating the unique hash value for the TD CCORE module name (eg. changes to this directive value will cause the CCORE to be re-generated).
- **OPT_CONST_DIVS** (*new directive*). By default, Catapult optimizes dividers driven by a constant. This directive provides control about whether Catapult performs this optimization.
- **PACKING_MODE** (*default changed*) The default value for this directive changed from *absolute* to *compact*.
- **STRUCT_SPLIT_MODE** (*new directive*): When a design contains structs on the interface, you can control how Catapult netlists the structs using the PRESERVE_STRUCT and STRUCT_LAYOUT directives. In this release, the STRUCT_SPLIT_MODE directive was added to provide additional control. For more information on this flow, refer to Automatic Splitting of Struct/Array datatype Interface ports.

Option Changes

The following list describes the changes to the Catapult options (Tools > Set Options...):

- **AprisaRTL Flow.** This new flow allows you to synthesize the RTL netlist generated by Catapult in the AprisaRTL synthesizer tool. This flow also allows you to characterize Catapult libraries using AprisaRTL. For more information on this flow, refer to Added support for the AprisaRTL flow (Beta).
- **Waveform database format** (*CDesignChecker, CFormal SLEC, and CFormal Stall flows*). When the associated flow detects a difference, it generates difference waveforms that allow you to reproduce the issue in simulation. Catapult Formal can generate difference waveforms in the *qwave* and *fsdb* database formats. The `WAVE_FORMAT` option denotes the default file format of the generated wave file. By default, the value is *qwave*.
- **Optional job launch command for <flow> invoke** (*CFormal Assert, Idle, IMP, SLEC, and Stall flows*). The `GRID_LAUNCH` option allows you to set the Linux command for invoking CFormal on your server grid. By default, this option is *blank*.
- **Site-specific command to launch a grid job** (*CFormal SLEC, and Stall flows*). The `GRID_WORKERS` option specifies a string to denote the setup configuration for the solver grid machines. This option sets the `host_setup_configuration` global parameters in the Catapult-generated run script.
- **Show the source code context in report** (*CFormal IMP*) The default value for the `SHOW_SOURCE_CONTEXT` option changed from *false* to *true*.
- The `ENABLE_COVERAGE_MAPS` and `ENABLE_PATH_BY_PATH` options were removed from the CFormal SLEC flow.
- **Monotonize ASIC Catapult Library Data** (*Component Library*). This new option informs Library Builder to adjust the characterized Area values to ensure that they increase (or stay the same) as the clock period decreases. This adjustment allows the scheduler to make better improvements. The original characterized data is stored in the library and can be restored. By default, this option is set to *true*. For more information on this flow, refer to Automatic smoothing of ASIC Library Data.
- **Write out mapping report** (*Design Compiler Flow*) The default value for the `ReportMapping` option changed from *false* to *true*.
- **Enable Enhanced Power Analysis option** (*LowPower flow*) If set to *true*, Catapult Ultra will use Enhanced Power Analysis (PowerPro's `-pa` switch) instead of its regular analysis (`-analyzer` switch). Enhanced Power Analysis requires Questa's 'gcc' compiler and the Oasys synthesis tool. By default, Catapult uses the `QuestaSIM/Path` option to locate the Questa install tree. Catapult uses the Oasys executable installed in the Catapult install tree. By default, this option is disabled.
- **Error Message Override** (*Message Options*). The `MEM-60` code was removed from the default list.
- **Set VIRTUAL_IO attribute true for all ports** (*Radiant flow*) When the `SetAllPortsVirtual` option is enabled (default), Catapult sets the `VIRTUAL_IO=TRUE` attribute on all top-level ports in the design in the Physical Design Constraint (.pdc) file. Catapult writes this file to the solution directory (eg. `rtl.v.pdc`). Radiant uses this option to place/route a block without using the IO cells.
- **RTL Manifest Flow.** When enabled, this flow automatically generates a RTL Manifest file for the Catapult-generated RTL file list. This optional flow is useful for passing RTL library and filenames in

user-defined format to downstream synthesis and simulation tools. For more information on this flow, refer to Added RTL Manifest flow.

- **Automatic STALL_FLAG toggle insertion** (SCVerify flow). The default value for this option has changed from *false* to *true*.
- **Transaction IO Sync override pairs (var:rsc sync:rsc ...)** (SCVerify flow). By default, SCVerify uses the transaction done (triosy) to determine when to capture data. This option allows you to override the transaction done (triosy) signal for non-channel inputs and outputs. The option accepts a list of IO resource pairs. The option can specify multiple pairs of resources. For each pair, SCVerify will replace the triosy for the first item with the second item's control signals (ie. the rdy/vld of an ac_channel or ac_sync). The override is only allowed on non-channel resources like scalar vars, memories, DirectInputs, etc and the replacement IO type must be a channel or a sync.

```
flow package option set /SCVerify/IO_SYNC_OVERRIDES {/top/mem:rsc /top/  
mem_sync:rsc}
```

The pairs can be either the resource HPATH (/top/mem:rsc) or just the name of the resource without the <top>, i.e. (mem:rsc). (Use the architecture constraints like MAP_TO_MODULE for the exact resource names).

- **Use time() to set rand() seed for SCVerify stalling** (SCVerify flow). If auto stall insertion is enabled (eg. AUTOWAIT set to non-zero value), then SCVerify, by default, uses a fixed value for the random seed to determine the number of clock cycles to wait during automatic stall testing. The maximum number of clock cycles is set by the AUTOWAIT_*_CYCLES options. This approach produces a consistent set of clock cycle waits during each simulation run.

If this option is set to *true*, SCVerify will use *time()* to set the random seed. As a result, the random stall intervals will not be consistent between simulation runs.

- **Stall hold count. Each stall is held for this number of clock cycles** (SCVerify flow). This option allows you to specify the exact number of clock cycles to stall during verification. By default, SCVerify holds the stall for 5 clock cycles.

Command Changes

The **solution export** command added an *adviser* option to the *-type* switch.

Pragma Changes

The **hls_idle** pragma was added. This pragma allows you to specify an IDLE signal in your SystemC design. This pragma has no affect on C/C++ designs. For C/C++ designs, use the IDLE_SIGNAL directive. For more information on this pragma, refer to IDLE signal logic generation for SystemC designs.

Corrected Problems

The following 100 customer-reported problems were fixed in this release.

- CAT-37917 The array read/write behavior is not as expected when using `ac_interleave_array_1D`
- CAT-37916 `ADVANCED_DPOPT = true` | Crash with segmentation fault at instance
- CAT-37915 `CCS_BLOCK` uniquification is not documented
- CAT-37902 Errors in SCVerify w.r.t template of `capture_IN/OUT` in `ccs_block_macros.cpp`
- CAT-37825 Support a seed for random stall cycle in SCVerify
- CAT-37682 SCVerify `nb_read` enhancement - first cycle for a channel can not be empty if later data
- CAT-37681 SCVerify `nb_read` enhancement - all inputs can not be empty
- CAT-37672 Document that SCVerify doesn't support unnamed nested structure on the interface
- CAT-37659 CDC Warning for Hierarchical Design with `AC_CHANNEL`
- CAT-37619 Simulation mismatch for single call for a ccore with static variable.
- CAT-37562 ECO: # Error: internal assertion failed (0) `sif_eco.cxx` line 688 (ASSERT-1)
- CAT-37557 Add file I/O functionality to `ac_int`
- CAT-37533 request for `link_library` option in memory generator to link memory .db file
- CAT-37531 internal assertion failed (`csteps(index) == csteps(tmpSched)`)
`sif_sc_sched.cxx` line 10945 (ASSERT-1)
- CAT-37447 Segmentation fault when setting "EmbedSrcFileRefs true"
- CAT-37421 final area report is not coherent with BOM when design contains `write_slice` operator
- CAT-37412 X_PHD: internal assertion failed (`csteps(index) == csteps(tmpSched)`)
`sif_sc_sched.cxx` line 10525 (ASSERT-1)
- CAT-37374 issue using `ac_int` VRA on design
- CAT-37310 Catapult is not sharing `write_slice` operator
- CAT-37279 `ac_window_v2_lflush` QofR Tests: Class vs Function Correlation for Toolkits
- CAT-37259 Connections header is not compatible with Accellera SystemC 2.3.1 or newer
- CAT-37246 `ac_window_v2_lflush` QofR Tests: Class vs Function Correlation for AE Testcases
- CAT-37245 AC Window 2.0 - Line Flushing- QofR Improvement
- CAT-37167 Rocky 8 Support Catapult 2024.1 + QuestaSim

- CAT-37161 latest 2023 versions salt_license_server is unsupported. only work w/ mgls_license_file
- CAT-37082 update lab documentation results adjusted to new version of Catapult
- CAT-37078 Add VRA support for ac_int - Implementing Feedback for Early Release
- CAT-36879 Check Analyze -vs- Compile runtime - analyze 3x compile
- CAT-36849 Add VRA postprocessing for ac_int
- CAT-36813 Update ac_sc.h with trace callback target
- CAT-36687 Removing Unnecessary Initializations in ac_window_v2_lflush
- CAT-36617 Optimizing float*float = fixed
- CAT-36612 insufficient resources when input array is read from 2 different if-else constructs
- CAT-36607 Optimizing float*float = float - Early Release
- CAT-36606 Area mismatch of approximately 5x between Catapult Estimate and DC
- CAT-36601 High runtime in Design starting from go assembly
- CAT-36594 Add VRA support for ac_int
- CAT-36586 Optimizing float*fixed = fixed
- CAT-36586 Optimizing float*fixed = float
- CAT-36549 Internal Assertion
- CAT-36538 Scheduler cannot schedule 0.66 ns memory read within 0.8 ns
- CAT-36530 Long runtime in DPFSM
- CAT-36472 CCore Issue with Block adder tree
- CAT-36454 Catapult crashes in compile; For array of pointers to ac_channel when nb_write is used
- CAT-36413 Revise ac::bfloat16 default rounding behavior implementation of either AC_TRN_ZERO/AC_RND_CONV
- CAT-36404 HLS design generation from pytorch - and support bias=False
- CAT-36378 Implement Verilog split port field wrapper in the core
- CAT-36333 High Runtime in go extract; GUI freezes but tool still runs till completion

- CAT-36326 about the (one-hot) mux delay estimation
- CAT-36315 Improve rtl coverage for ac_div.h
- CAT-36302 Need a function for checking if ac_float input value is normalized or not
- CAT-36297 Case of potential RTL synthesis mismatch
- CAT-36194 Adder tree flattening is sub-optimal with pragma based clustering
- CAT-36185 Productize Verilog split port field wrapper
- CAT-36088 Need ac_float type conversion warnings/errors issued by g++ compiler resolved
- CAT-36087 Optimizing float*fixed = float - Coding and Testing
- CAT-36002 Implement VRA (Phase I) for ac_int
- CAT-35944 Put auto_gen_port_info.h and auto_gen_fields.h at one place
- CAT-35937 Simulation results look like WBR when Xilinx_RAMs.BLOCK_1R1W_RBW is used
- CAT-35809 ac_window_v2_2d_lflush- Schedule failure when non power of 2 linebuffer arrays used
- CAT-35742 Document that VRA doesn't track ac_fixed variables in file names beginning with "ac_"
- CAT-35670 SCVerify capture memory packets using a helper signal
- CAT-35649 Custom FIFO/RAM RTL import (Memory Generator/FIFO Generator)"
- CAT-35622 peek() and nb_peek() - Phase 1
- CAT-35560 SoC Design - AXI-Lite - phase 3
- CAT-35527 Add FSDB/qwave support option to all/each cformal app flow setup.
- CAT-35518 CFormal: Debug makefiles need background launches
- CAT-35463 Update info for Assertion Synthesis
- CAT-35395 SystemVerilog netlist generation enhancements
- CAT-35367 AC Window 2.0 Line Flushing Interpolation Support
- CAT-35320 Documentation around CCOREs with INOUT ports is incomplete
- CAT-35261 HLS4ML Phase 2

- CAT-35189 Make CCS_BLOCK macro line independent
- CAT-35095 Request Support for Avant-E Family of Lattice FPGAs
- CAT-34924 Update connections.h to use DIRECT_PORT by default for pre-HLS simulation
- CAT-34790 Support for TKEEP, TSTRB, TID and TDEST in AXI4-Stream
- CAT-34161 CFormalAssert examples document: add TCL control file list
- CAT-34026 Extrapolation issue with large mux
- CAT-33598 Long runtime when IOREAD is embedded inside a loop body
- CAT-33455 Update documentation to enable OTF Flow with DC Topo mode in Catapult
- CAT-33108 Testbench Clock not overridden by Clock periods defined by PWR_CLOCK_STATES during Power Estimation Sim run when using SystemC
- CAT-33079 Builtin_modularIO Support details are not clear in Reference Manual
- CAT-32806 Sif assert in go analyze
- CAT-32780 SCVerify should let user to keep valid low for unused input throughout the simulation
- CAT-32766 SCVerify should be able to control input's vld based on testbench when throughput=1
- CAT-32186 CRD-ERR - Compilation errors in include libraries- 'basic_string.h(6001)' & 'stl_pair.h' when c++ standard set to 17 (-std=c++17)
- CAT-31362 Need fix for Liberty file parsing issue in Catapult Library Builder
- CAT-30913 DA adviser print/export support; cmd line invocation of DA adviser-generate reports
- CAT-30793 Support FWFT Peek+Read Interfaces
- CAT-30254 Add support for register_file RAMs for FPGA
- CAT-29269 Change color of read_stall and write_stall ccs_pipe signals in SCVerify waveforms
- CAT-27661 Examples of pragmas/directives for CCores in Class- and Function-based design
- CAT-26508 "Failed schedule with ""not enough resources"" with II=1 for particular coding style"
- CAT-26244 Applying modified res_sharing.tcl after "go dpfsm" causes tool to hang
- CAT-25902 Need a reg based FIFO (pipe component) with dout_rdy to din_rdy path registered
- CAT-25794 SCVerify with VCS-MX causes Stack Overflow (QuestSIM works)

- CAT-24328 Catapult generation of something like a 'catapult_netlist.f' file
- CAT-24061 Unnecessary FIFO_DEPTH directive is generated by GUI operation
- CAT-23305 Requesting Memgen to support system verilog files
- CAT-5557 Request to allow external memory with mgc_pipe_mem channels

Release 2024.1_3

This release provides bug fixes since the 2024.1_2 release. No directives, options or commands were added, removed or changed in this update release.

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-38107 output/CaseMux true creates incorrect RTL in mux's pessimism reduction
- CAT-37619 simulation mismatch with single call for a ccore with static variable.
- CAT-37562 ECO: # Error: internal assertion failed (0) `sif_eco.cxx` line 688 (ASSERT-1)
- CAT-37412 X_PHD_SYNTHESIS: Error:internal assertion failed `csteps(index)==csteps(tmpSched)` `sif_sc_sched.cxx` line 10525 (ASSERT-1)
- CAT-37410 dataflow `sif_assert` (`sif_df_vstat.cxx` line 219 and 223), related to UserOper
- CAT-37399 simulation mismatch with FF merge happening before register removal step
- CAT-36538 Scheduler cannot schedule 0.66 ns memory read within 0.8 ns
- CAT-36454 Catapult crashes in compile for array of pointers to `ac_channel` when `nb_write` is used
- CAT-36297 Case of potential RTL synthesis mismatch

Release 2024.1_2

This release provides bug fixes since the 2024.1_1 release. No directives, options or commands were added, removed or changed in this update release.

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-37250 Simulation mismatch with const ROM array opt if DA_DISABLE_RESIZE_MEM=false
- CAT-37166 # Error: internal assertion failed cdfg/sif_cd_convvals.cxx line 1209 (ASSERT-1)
- CAT-37136 Unexpected constant logic is generated according to coding style in SystemC flow
- CAT-37053 Empty loops are not optimized out, they were optimized in 2022.2 version
- CAT-36994 Update/Fix for PowerPro 2023.2_5 for Catapult 2024.1_2 release
- CAT-36955 Additional PowerPro request for genvar for Spyglass Lint
- CAT-36910 Update PowerPro to 2023.2_5 for Catapult 2024.1_2 release
- CAT-36900 Catapult crashes with internal assertion error (ASSERT-1)
- CAT-36889 Assertion failure during allocation stage in Class based hierarchy design
- CAT-36871 Spyglass Lint Error: W121 Warning- genvar variable not unique
- CAT-36863 DesignAnalyzer misses critical path information
- CAT-36856 simulation mismatch when a member function defined as a CCORE
- CAT-36542 RockyLinux OnTheFly Error
- CAT-36072 Scverify issue InterCost_Init
- CAT-36065 design gets stuck at scheduling forever
- CAT-35505 W159 spyglass warnings
- CAT-35470 catapult fifo sizing issue

- CAT-28621 Improve error message for incompatible FIFO_DEPTH

Release 2024.1_1

This release provides bug fixes since the 2024.1 release. No options or commands were added, removed or changed.

Enhancements/Changes

Prior to 2024.1, Catapult preserved unused DirectInput Ports. In the 2024.1 release, Catapult optimized these ports away.

In the release, by default, unused DirectInput Ports are preserved (restoring the pre-2024.1 functionality). The new `REMOVE_UNUSED_DIRECT_INPUTS` directive provides users with control over how Catapult handles this situation. If the directive is changed to *true*, any unused DirectInput ports will be optimized away.

```
directive set /REMOVE_UNUSED_DIRECT_INPUTS true
```

By default, this directive is set to *false*.

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-36643 `sif_assert` in `sif_lt_compat.cxx` (DPFSM)
- CAT-36641 Logic is optimized away when unrolling a loop in the `ac_math::ac_sincos_cordic`
- CAT-36544 GUI transcript window not transcribing after **solution netlist** post **go instance**
- CAT-36532 Need a directive to preserve unused direct inputs
- CAT-36529 Catapult not erroring out by default for an INOUT port on a comb. CCore
- CAT-36482 Need deferred pragma evaluation
- CAT-36450 `reuse_factor` does not affect `dense_latency()`
- CAT-36416 Syntax error in VHDL `ccs_dw_*` components
- CAT-36403 improve hls4ml flow to report FIFO and weight/bias ROM info
- CAT-36363 Different RTL generated with same input files

- CAT-36341 Catapult LB 2024.1 is using Catapult 2023.2 for characterization.
- CAT-36318 Unable to select amba.ccs_axi4stream_in for channel with nb_read
- CAT-36305 SCVerify - add warning at end of capture() when there is unconsumed data left in the C++ channels
- CAT-36296 Failed Clustering
- CAT-36180 Design stuck running in go extract

Release 2024.1

The following topics describe the changes since the 2023.2_2 release. The Changes to Directives/Options/Commands/Pragmas section describes all new, removed or changed Directives and Options. The following sections describe the enhancements that were added since the last release.

Enhancements/Changes

The following sections describe the enhancements that were added in this release:

- Improved Logic Optimization
- Improved Timing Estimates for chained Arithmetic Logic
- New class for Array Handling (ac_bank_array)
- Support for HLS4ML flow (beta)
- Flushing Support for AC_WINDOW_V2
- Shared Memory Port Reservation
- ASIC Library Enhancements

Improved Logic Optimization

In this release, the new LOGIC_SYNTHESIS directive enables enhanced logic synthesis to perform logic optimizations on the design, and consequently estimate the delay and area cost of logic operations more accurately. When enabled, these optimizations allow Catapult to report a more accurate final area and delay with respect to downstream synthesis tools.

Note: This directive can only be set using Catapult Ultra. If you are using Catapult HLS, the directive can not be changed from its default value (no).

The LOGIC_SYNTHESIS directive must be set prior to the *compile* state:

```
directive set /LOGIC_SYNTHESIS {no | estimation}
```

Where:

- **no** – disables the logic synthesis feature (default)
- **estimation** – enables the logic synthesis feature for computing the optimized area/delay estimates

For multiple examples of the various optimization enhancements produced by this new directive when enabled in Catapult Ultra, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > LOGIC SYNTHESIS**).

Improved Timing Estimates for chained Arithmetic Logic

In previous releases, when several arithmetic operators were concatenated or chained together, Catapult would overestimate the timing through these datapaths. Because Catapult considered the timing model of chained operators as the sum of the individual delays, i.e. a rectangle or a word-wise timing model, it ignored the difference delay values through the MSB and the LSB of the result of chained operations. The LSB arrives much earlier than the MSB, unlike the rectangle model which assumes that all bits arrive at the same time. In other words, they have a parallelogram timing model or a bit-wise timing model.

In the 2023.2, this new timing model was used for reporting estimates.

In this release, this new timing model can be used by the scheduler.

For multiple examples of the improvements due to this new timing model, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Advanced Timing Model (schedule)**).

New class for Array Handling (**ac_bank_array**)

Currently, High-Level Synthesis allows for describing the memory required by an algorithm using a C-style array that HLS maps to a physical memory component during Interface Synthesis. The user can apply **BLOCK_SIZE** and **INTERLEAVE** constraints in the Interface Synthesis step that allows the C-style array to be "reshaped" to different physical structures.

However, care must be taken to ensure that the results of these constraints align with a multiple of the array dimension which you are attempting to partition the memory. And if the dimensions are not a power of 2, the resulting address logic may not be optimal and the HLS runtime may increase due to the overhead of the unexpected address logic.

In this release, the new **ac_bank_array** and **ac_interleave_array** classes provide an approach to improve your QofR and reduce the HLS runtime when you want to create hardware with a pre-HLS array mapped to banked physical memory instances.

In order to address these QofR/runtime issues and provide a hint in the pre-HLS source as to the desired memory banking structure, two new classes **ac_bank_array** and **ac_interleave_array** have been added to the Catapult datatypes. The classes in **ac_bank_array.h** give the user control over the memory bank structure of an array during synthesis. The classes in **ac_interleave_array.h** give the user control over the interleave factor for an array during synthesis.

For multiple examples of the various optimization enhancements produced by this new directive, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Banked and Interleaved Memories**).

Support for HLS4ML flow (beta)

In this release, the new HLS4ML flow provides an integration of the open-source HLS4ML package within Catapult HLS. The HLS4ML package provides a method for taking data from trained ML models using open-source packages (eg. Keras) and generating C++ code that is synthesizable by Catapult HLS. The flow allows you to configure the HLS to best meet the needs of your algorithm.

In order to run the HLS4ML examples, you must execute the following Linux commands to patch the `ccs_venv` that the 2024.1 release currently creates:

```
cd $HOME
bash
source $HOME/ccs_venv/bin/activate
pip install tf_keras
pip install onnx
pip install pyDigitalWaveTools
```

After doing this, you should be able to run the HLS4ML toolkit examples.

For an overview of the flow along with multiple examples of using the Beta version of this new flow, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > HLS4ML**).

Flushing Support for AC_WINDOW_V2

In the previous release, the new AC Window 2.0 classes provided a number of new features such as the ability to process multiple data inputs in parallel, more line buffer memory architectures to choose from as well as enhanced modularity.

In this release, AC Window V2 added flushing support classes. For examples of using the flushing and line flushing classes in AC Window V2, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > IPL (ac_ipl) > ac_window_v2 > ac_window_v2_2d_*flush**).

Shared Memory Port Reservation

In previous release, Catapult added support for Shared Memories and the associated synchronization using `ac_shared` and `ac_sync`.

In this release, Catapult introduces the `MEMORY_USE_PORT` directive. This directive controls the memory port of a multiport memory that a process/thread can use to access a specific shared memory. (Single port shared memories will no longer be supported). Not setting the directive can lead to port conflicts when the design is assembled. The directive has the same effect in bottom-up and top-down. In bottom-up, there will not be any errors when synthesizing the block, but the block may not be usable in the system later.

It can only be omitted safely if the usable memory ports can be inferred from the actual use, e. g. for a separate rd/wr port memory with one block only reading and the other block only writing the memory.

The directive is set on the resource object of a specific process/thread. It will typically be the interface pointer that the block uses to access the memory. The constraint is not propagated to all connected resources, because typically all process/threads need different setting to avoid the use of conflicting ports. It only affects interface resources on process/threads that actually access the memory. Setting it on the actual array resource has no effect.

The value is a TCL list of ports to access a memory. Each list element selects one port that a process/thread can use. More list elements can increase the throughput of a process/thread. Each list element is a plain number or a pair of {<direction> <position>}

- 2
use second port (can be read, write, or rdwr) (error if direction does not match the access)
- {1 3}
use first and third port (error if direction does not match the access)
- {{rd 1}}
first read or rdwr port
- {{rd 1} {wr 2}}
use first read/rdwr port, and second write/rdwr port (error if both refer to the same rdwr port)
- {rd 1}
Error: missing {} when single element can be misinterpreted as a list

The following pseudo-code example shows a simple design with a shared memory within the *reader* and *writer* blocks:

```
class writer_c {
public:
    writer_c() : RAM_select(true) {}

    #pragma hls_design interface
    void CCS_BLOCK(run) (ac_channel<data_t> &din,
                        ac_shared<data_t[64]> &pingPong,
                        ac_sync &sync_RAMs)
    { ... }
};

class reader_c {
public:
    reader_c() : RAM_select(true) {}

    #pragma hls_design interface
```

```
void CCS_BLOCK(run) (ac_shared<data_t[64]> &pingPong,
                    ac_sync                &sync_RAMs,
                    ac_channel<data_t >    &dout )
{
    {...}
};

#pragma hls_design top
class dut_c
{
    writer_c writer;
    reader_c reader;
    ac_shared<data_t[64]> pingPong;
    ac_sync sync_RAMs;
public:
    dut_c() {}

    #pragma hls_design interface
    void CCS_BLOCK(run) (ac_channel<data_t > &din,
                        ac_channel<data_t > &dout) {
        writer.run(din,pingPong,sync_RAMs);
        reader.run(pingPong,sync_RAMs,dout);
    }
};
```

For this design, you could assign the writer to port 1 and the reader to port 2 using the following directives:

```
directive set /dut_c/writer_c/pingPong.d:rsc -MEMORY_USE_PORT 1
directive set /dut_c/reader_c/pingPong.d:rsc -MEMORY_USE_PORT 2
```

For multiple examples of using this new style for Shared Memories in various configurations, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Shared Memories Synchronized Access**).

ASIC Library Enhancements

This release provides two enhancements to increase the accuracy of library operators within Catapult:

- **Improved interpolation equations.** After Library Builder gathers the characterization data from the RTL synthesis runs, it applies an interpolation equation to generate area and delay equations for a given operator. Catapult uses these generated equations during optimization and scheduling.. In previous releases, there were a few cases when these generated equations may incorrectly yield an area or delay value of zero.

In this release, the interpolation equations were adjusted to avoid these situations. To utilize these new interpolation equations, you currently need to create a new library (to acquire the new equations from the ASIC base library template) and then apply the TCL file containing your characterization data.

- **Improved Iterative Estimator.** In previous releases, Library Builder used a simple iterative process to determine the area/delay of the 'fastest' implementation for Qualified Module (QMOD) for arithmetic operators (eg. `mgc_add`, `mgc_mul`, `mgc_mul_pipe`). This was the first version of the Iterative Estimator for Fastest (*ieff*). Although this initial approach typically produced better results than simply over-constraining the clock using the default characterization clock period of 0.001ns, it did not yield improved results for non-arithmetic operators.

In this release, Library Builder provides an improved version of the iterative estimator (*ieff_version=2*) that works for all operators in the Catapult library and provides improved results for arithmetic operators compared to the initial iterative estimator. To utilize the newest version of the iterative estimator, add the following four variables to your Catapult library and re-characterize the operators.

```
Library add /<lib>/VARS/ieff_version 2
Library add /<lib>/VARS/ieff_slack_multiplier::0.75
Library add /<lib>/VARS/ieff_iter_max::9
Library add /<lib>/VARS/ieff_wns_target::0.000
```

If you are generating a new Catapult library, these variables will be automatically added to the library. You just need to change the *ieff_version* variable from '1' to '2'.

Changes to Directives/Options/Commands

Directive Changes

The following list describes the changes to the Catapult directives.

- **CCORE_INOUT_MODE** (*new directive*) In previous releases, CCOREs containing INOUT ports with partial/conditional outputs were split into separate in and out ports and possibly generated logic that the user may not intend. The new **CCORE_INOUT_MODE** directives allows Catapult to consistently handle CCOREs with INOUT ports and issue informational message on how Catapult is handling the INOUT port in the RTL netlist. This directive takes one of the following values:
 - *auto* (default) True inouts will be split automatically while inouts with partial/conditional outputs will error
 - *split_io_port*: Catapult will split the INOUT into separate in and out ports if and only if an output is identified as partial/conditional. This setting will overcome the error from *auto* and instruct Catapult to achieve inout behavior via split I/O ports.
 - *output_only*: This is a new enhancement to CCORE handling, and user can avoid split io and avoid error, if the usage context is amenable. As of 2024.1, this value is only available as an early alpha option.

Whenever port splitting or forced output happens, appropriate warning message is issued identifying the port and the CCORE and the reason for such behavior. The default value for this directive changed from 0 to 4. As a result, when Catapult Ultra encounters a combinational CCORE, Catapult will generate four implementations.

- **CCORE_POINTS** (*default changed*) The default value for this directive changed from 0 to 4. As a result, when Catapult Ultra encounters a combinational CCORE, Catapult will generate four implementations.
- **LOGIC_SYNTHESIS** (*new directive*) This directive informs Catapult to perform additional logic optimizations to improve the correlation between Catapult's area/delay estimates and the downstream RTL synthesis results. Refer to Improved Logic Optimization.
- **MEMORY_USE_PORT** (*new directive*). This directive provides control over which port of a multi-port memory that a process/thread can use to access a specific shared memory. Refer to Shared Memory Port Reservation.

Option Changes

The following list describes the changes to the Catapult options (Tools > Set Options...):

- **Source properties used as assumes and not checked** (*Cformal SLEC Flow*). This new option informs the Catapult Formal tool to evaluate source properties as assumes with running a formal check.
- **HLS4ML flow** This [new flow](#) is in BETA for this release.
- **Enable Visualizer for QuestaSIM GUI** (*QuestaSIM Flow*). This new option informs Catapult to use Visualizer instead of the QuestaSIM GUI when viewing SCVerify results. If enabled, the flow will automatically generate the required *design.bin* using *vopt*.
- **SpyglassLint Rule Parameters** (*Spyglass Lint Flow*). The following parameters were added to the default value for this option:

```
nocheckoverflow W416    maxLoopIter 32
handle_large_bus yes    ignore_if_case_statement yes
```

- **SGDC File Paths** (*Spyglass Lint Flow*). This new option specifies a string with a space-separated list of SGDC file paths. If specified, Catapult adds the "read_file -type sgdc " command to the generated script file for Spyglass Lint. If the SpyGlassLint_SG_SGDC_FILE environment variable is set, Catapult gives precedence to the environment variable value (even if it is empty).
- **SV Interface flow** This new flow generates a SystemVerilog wrapper for the top-level RTL Verilog design.
- **Scrollbar Limit** (*Message*) This new option allows you to limit the number of lines retained in the Catapult GUI Transcript window after a command is executed. In some cases, the transcript window content could become quite large and require excessive memory resource (eg. running simulation for power data). By default, this option limits the number of transcript lines after a specific command to 5000. This option does not affect the contents of the Catapult logfile.
- **Embed Pre-HLS source file references** (*Output*). If this option is enabled, the RTL netlist will include the C++ source file/line number for the modules and always-blocks in the Verilog and entity and

process blocks in the VHDL. The C++ source file reference in the netlist will include a 'Pre-HLS File: ' sub-string in the netlist.

Command Changes

The **project load** command added a *-solution* switch. In previous releases, the last solution was automatically selected after the project loaded. This new (optional) switch allows you to specify which solution Catapult should initially select after the project loads.

The **catapult** shell command added a new *cformal* option to the *-product* switch. When using the *-product cformal* switch, Catapult requires a CFormal license (catapultFormal_c) and will only run through the *compile* stage. At the end of the *compile* stage, you would be able to run CDesignChecker or the CFormal SLEC flow. If you run the 'solution library add' command (eg. executing an existing Catapult HLS script), Catapult will generate an expected error during the 'libraries' stage when loading the library because libraries can only be read after the 'compile' stage. In addition to running this product option using the GUI or Tcl commands, you could read an existing project and run any of the CFormal applications and CDesign Checker.

Corrected Problems

The following customer reported problems were fixed in this release.

- CAT-36066 Design with single sub-block with inout port is optimized away
- CAT-35968 Invalid component selection '{[Register]}' for Resource '/sc_reg_array/run/R:rsc'
- CAT-35905 output port missed in ccore
- CAT-35876 Doc updates for the ROM_CLU
- CAT-35845 Improve ccs_pipe_pointer toolkit example
- CAT-35842 Compilation error on Questa/VCS/DC/Oasys for the SV RTL
- CAT-35828 PowerPro work dir overwritten
- CAT-35816 Catapult Mem generator : Unknown path Error When GEN_RAM_PIPE = 1
- CAT-35808 Clustering Fails abruptly
- CAT-35789 SystemVerilog package is not generated well; SCVerify Errors in SystemC flow
- CAT-35773 Add CatapultFormal_c to release registration
- CAT-35760 Warning message from pragma hls_resource in SystemC flow
- CAT-35758 Excessive DPFSM runtime for Seq CCORE with no output reg and latency of 1

- CAT-35753 Update supported platform to Rocky 8.9
- CAT-35737 HLS4ML examples need license CatAlhls4ml
- CAT-35668 Rewrite locked reloadable SIF object
- CAT-35656 SV 'logic' datatype not being used for all wires/reg.
- CAT-35641 Cross probe on CFA assert not working.
- CAT-35636 catapult report compile aborted issue but not report the reason
- CAT-35634 SV always_ff with async reset has iff in sensitivity list.
- CAT-35608 Netlist packaging and prefixing is inconsistent with structs on the interface
- CAT-35593 Branch Change always ask option setting is not working in shell mode
- CAT-35587 auto_gen_port_info.h: Incorrect array dimension order in wrapper rtl
- CAT-35586 HLS4ML examples are missing in ALPHA1
- CAT-35547 Sim mismatch in 2023.2_2;not 2022.2: Inputs tied to 0 in all blocks; only 2 intended
- CAT-35546 Pass-by reference input to function mapped to CCORE loses value
- CAT-35524 Clustering fails when CFormalSLEC is enabled (cluster-2 error)
- CAT-35491 sif_cd_deadcode.cxx : Internal assertion failed (sellen)
- CAT-35490 RTL sim mismatch due to potential logic fault
- CAT-35483 CFormal IDLE Makefile collateral not generated when using SV style output
- CAT-35474 sif assert / sif_cd_tempelim.cxx at dpfsm followed by a Catapult Crash
- CAT-35460 SCVerify makes wrong wrapper for sc_in<bool>
- CAT-35377 CFormal apps need to handle STRUCT_LAYOUT usage
- CAT-35369 systemverilog package is not included in generated RTL from SystemC design
- CAT-35358 Fix management of alib-52 in Library Builder so it works on the Farm
- CAT-35349 SCVerify/VCS-MX # Error-[SV-LCM-PND] Package not defined
- CAT-35314 sif_nl_name_dict.cxx : Internal assertion failed (revstri == revNameMap.end())

- CAT-35308 #Error: internal assertion failed (p_.first_pass_)
- CAT-35282 Request SV style constructs for procedural assignments in generated RTL
- CAT-35273 VCS fails to compile Memory Generator's SystemC RAM model
- CAT-35268 Use 'logic' datatype for the generated SV style RTL
- CAT-35251 Mitigations for critical QOR problems in SystemC preserve_struct=true
- CAT-35236 SV style interface wrapper
- CAT-35198 Array of pointers to ac_channels coding style: Design disappears using private method based hierarchy
- CAT-35196 Missing multi-dimension arrays in FSDB for power in case of Verilog only
- CAT-35192 AC Window 2.0 Line Flushing (No Interpolation) - Phase 6: Toolkit Docs
- CAT-35190 Sm mismatch RTL from Catapult: Writer module gets removed in assembly when specified as private member method based hierarchy
- CAT-35181 Add preHLS source file/line number to module/entity and always/process declarations in HDL netlist
- CAT-35175 Document VHDL component package support for memory generator
- CAT-35166 Req: Remove CCore hash from connected wires at instantiation.
- CAT-35162 Name conflict in Catapult VHDL netlist with DirectOutput
- CAT-35134 ASIC low power pipe component
- CAT-35112 2023.2_1 onwards design using Combo ccore starts failing w/INOUTs are not supported for CCOREs
- CAT-34980 Run CFormal Apps from catapult -prod cformal
- CAT-34979 sif_assert at go assembly while running an FSM code in C++
- CAT-34969 Another sif_assert found when testing for AMD
- CAT-34953 Top-down: Catapult errors; Bottom-up: Catapult sif_assert(s)
- CAT-34941 Assertion failure due to missing ports in block after BLOCK_SIZE
- CAT-34936 Using FORCE_AUTO_PORT=Connections::DIRECT_PORT breaks the Connections tracing

- CAT-34934 Update docs with new ac_blackbox features
- CAT-34932 AC Window 2.0 Line Flushing (No Interpolation) - Phase 5: Reference Manual
- CAT-34931 AC Window 2.0 Line Flushing (No Interpolation) - Phase 4: 2D Toolkit Synthesis
- CAT-34930 AC Window 2.0 Line Flushing (No Interpolation) - Phase 3: 2D Toolkit HLV
- CAT-34927 portname mismatch due to name conflict resolver in BIF code
- CAT-34923 ResetVal in .csv doesn't work for Bus Slave Gen
- CAT-34858 AC Window 2.0 Line Flushing (No Interpolation) - Phase 2: C++ Unit Test
- CAT-34836 Area increased by 1.4x for SystemC design in 2022.2 & 2023.2_1
- CAT-34789 Catapult inferring extra loops for reset process
- CAT-34772 Bus Slave Generation failed
- CAT-34765 Scheduling fails with 2023.2; succeeds with 2022.2
- CAT-34761 Unable to simulate on Xcelium with DesignWare library
- CAT-34753 DirectOutput not supported by UVM flow.
- CAT-34700 Catapult creates wrong RTL for specific SystemC SC_METHOD construct
- CAT-34582 Add support for DW_minmax component in std cell library template
- CAT-34570 Internal assertion failed (! this->get_array_dim()) sif_ci_dfg.cxx line 3589
- CAT-34569 Internal assertion failed (d_array.size() >= in1_arr->d_array.size())
- CAT-34544 Internal assertion sif_cd_tempelim.cxx when PRESERVE_STRUCTS=false; STRUCT_LAYOUT=legacy
- CAT-34537 Internal assertion sif_if_apply.cxx when PRESERVE_STRUCTS=false; STRUCT_LAYOUT=legacy
- CAT-34521 AC Window 2.0 Line Flushing (No Interpolation) - Phase 1: C++ Coding
- CAT-34470 CFormalAssert:link calypto->last run, reruns have lost sync to GUI
- CAT-34451 Update PowerPro release and verify spyglass failure
- CAT-34421 No error message if signal is missing from sensitivity list of SC_METHOD
- CAT-34410 Tool Crash during compile stage due to mapped

- CAT-34366 CFormalAssert Placeholder Assert-to-assume
- CAT-34359 FIFO generator doesn't have option for size port
- CAT-34352 DA taking very long to load during scope mapping
- CAT-34299 Add HLS4ML MVP
- CAT-34164 Dragging imaginary var onto same memory resource as real SIF asserts
- CAT-34111 Custom FIFO/RAM RTL import (Memory/FIFO Generator)
- CAT-34109 Byte write enable Support for FPGA RAM (BRAM/URAM)
- CAT-34087 Unused DirectInput is preserved when MAP_TO_MODULE is set with wildcard
- CAT-34045 sif_assert during dpfsm src/cdfg/sif_cd_constprop.cxx line 3481
- CAT-33888 SCVerify Failure
- CAT-33877 Add new ac_bank_array classes into ac_types/examples/documentation
- CAT-33617 add AC_WINDOW_v2 Flushing Support (no interpolation)
- CAT-33524 Documentation update for Vinit loop
- CAT-33518 Run CFormalAssert from catapult -prod cdesign_checker
- CAT-33508 IO SummaryTable incorrect for ccs_ctrl_in_buf_wait component
- CAT-33456 Catapult doesn't detect mutually exclusive memory access in unrolled loop.
- CAT-33394 Catapult OnTheFly not invoking DC in topographical mode when using Flow Option settings from DC FlowFile
- CAT-33299 Attempt to make KeyExpansion function into CCore gets stuck in 'go compile'
- CAT-33122 internal assertion failed pointing to Catapult src/classes/sif_gantt.cxx
- CAT-33076 hls_resource pragma doesn't work for SystemC connection resources like ccs_conn_ctrl_in_buf_wait/ccs_conn_in_wait
- CAT-32960 Uncompetitive runtime with Stratus in second run
- CAT-32916 Catapult crashes when ac_shared is used with 2 synchronous clocks
- CAT-32793 Sif_assert at go compile
- CAT-32337 Memory bloat and slow GUI after 'go switching'

- CAT-31813 Do not use fsdbDumpOff in low power flow with Questa
- CAT-31383 Create application note about the use of direct inputs
- CAT-31365 Negative Slack due to and/mux chain
- CAT-31346 GATE_REGISTERS affects Catapult HLS Results
- CAT-30959 Requesting DW FP support for exponential operator
- CAT-30439 Unnecessary FIFO_DEPTH directive to non-fifo interface component
- CAT-30153 ac_shared variable is unconnected from top-down to bottom-up block
- CAT-30064 Loop transform time increases exponentially if unrolled loop has multiple conditions
- CAT-29525 Improve timing and area estimation of concatenated operations
- CAT-28893 Seg-fault when Catapult not able to write to Cache
- CAT-28865 Direct Inputs are not direct
- CAT-28489 Contention in ccs_designware and sample baselibs
- CAT-28237 hls resource array to memory library failure
- CAT-26466 Top level design argument is interpreted as inout when it is pure a output
- CAT-26464 Area efficient ac_float adder tree
- CAT-23760 Class-based hierarchy with no pragmas
- CAT-13501 pragma hls_resource doesn't work with static local variables or class attributes
- CAT-12736 Passing Reference to an Array of Channels Causing sim mismatch in 'go assembly'
- CAT-9293 Clock gating not applied for registers driven by an AND gate
- CAT-7250 QofR issue with sharing nested muxing (could require mux-pushing)
- CAT-6712 Unresolved dependencies prevent scheduling of mutually exclusive singleport rams
- CAT-3644 Schedule fails with single port memory even with ignore_mem_precedence
- CAT-2849 Catapult not merging memory accesses when loops unrolled - addresses not seen as same due to concatenation

Release 2023.2_2

This release provides bug fixes since the 2023.2_2 release. No Directives were added, removed or changed. No new functionality was added in this release.

Enhancements/Changes

The following items changed in this release:

- The *hls_connections* library was updated from version 2.0.0 to 2.1.0.
- The AC Datatype library was updated from version 4.7.0 to 4.8.0.
- The SCVerify flow added a new option called *INIT_OUT_MEMORY_TRANSACTIONS*. When enabled, output arrays mapped to a memory will be initialized with the contents of the array in the *capture_IN* function. This flow does not support channels of structs with arrays mapped to memory on the interface because there is nothing to use for initialization. For backwards compatibility, this new option is disabled by default.

```
option set Flows/SCVerify/INIT_OUT_MEMORY_TRANSACTIONS false
```

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-35168 2023.2_1: .lock files in CCore directories of Cache have protections of owner RW only
- CAT-35153 Additional skid buffers causes regression
- CAT-35136 Create Catapult HLS AMD product configuration for Xilinx parts
- CAT-35083 CCOV runs removed from Matchlib toolkit example
- CAT-35082 boost library install path missing in examples
- CAT-34952 sif_assert during testing
- CAT-34941 Assertion failure due to missing ports in block after BLOCK_SIZE
- CAT-34927 portname mismatch due to name conflict resolver in BIF code
- CAT-34922 RTL vs C mismatch using CCORE

- CAT-34872 Documentation for connections incorrect
- CAT-34870 connections_fifo.h error: 'setw' is not a member of 'std'
- CAT-34824 sc_logic missing from mc_typedef_T_traits
- CAT-34616 Internal assertion sif_cd_resize.cxx: ROM cluster off in SystemC
- CAT-34593 Internal assertion sif_cl_netlist.cxx : SystemC CCORE synthesis
- CAT-34586 Combinational CCORE has much larger delay (and area) due to different logic implementation in current builds than what existed in 10.6a
- CAT-34535 Enhance SCVerify to initialize output memory transactor
- CAT-34229 Assertion failure and Seg Fault due to missing ports in sub-block instantiation
- CAT-33752 Interface Mismatch of a Bottom-up block, when select internal (with external pins)/externalized resources are optimized
- CAT-33524 Documentation update for vinit loop
- CAT-26932 ac_sc.h has non-portable code for vcd tracing (can cause crashes on some simulators)

Release 2023.2_1

This release provides bug fixes since the 2023.2 release. No Directives or Options were added, removed or changed. No new functionality was added in this release.

The *hls_connections* library was updated from version 1.5.0 to 2.0.0.

Corrected Problems

The following customer reported problems were fixed in this release:

- CAT-34595 Set PowerPro global `__wrtl_clk_lint_clean` default to ON
- CAT-34589 Internal error when using `ac_windows_v2_2d`
- CAT-34574 Matchlib toolkit examples in latest catapult release are missing important files.
- CAT-34561 Sif Assertion During CCORE Characterization with Design Compiler
- CAT-34544 Internal assertion `sif_cd_tempelim.cxx` when `PRESERVE_STRUCTS=false`, `STRUCT_LAYOUT=legacy`
- CAT-34537 Internal assertion `sif_if_apply.cxx` when `PRESERVE_STRUCTS=false`, `STRUCT_LAYOUT=legacy`
- CAT-34364 Memory generator pipe misses memory ports
- CAT-34153 Memory Generator creates incorrect SystemC memory model
- CAT-28766 `cat2slec` errors for `WORD-WIDTH` directive even if not specified

Release 2023.2

The following topics describe the changes since the 2023.1_2 release. The Changes to Directives/Options/Commands/Pragmas section describes all new, removed or changed Directives and Options. The following sections describe the enhancements that were added since the last release.

Enhancements/Changes

The following sections describe the enhancements that were added in this release:

- Improved Struct Type Interface Mapping
- New Timing Model for Reporting
- Value Range Analysis Tool
- Shared Memories using `ac_sync`
- AC Window 2.0
- Design Analyzer Improvements
- DirectOutput Resource Type
- Library Changes
- New CFormal Assert App (Beta)
- Bus Slave Generator Flow (Beta)
- Punch `ac::probe()` Signals to Top-Level (Beta)

Improved Struct Type Interface Mapping

In previous releases, Catapult would split Struct-type objects into leaf-level fields by default. While variables can be combined with architectural constraint directives, Catapult provides an ability to keep the struct-type `ac_channel` 'unsplit' using the directive `PRESERVE_STRUCTS=true`. Further, Catapult used flattened logic vector interfaces for `ac_channels` and SystemC signals, with a layout that enforced LSB to MSB assembly of contiguous field data, with a difference in term of how arrays are handle. For a C struct type, if there are no arrays or if the arrays are only present in leaf field, the generated layout will match the C-style layout of struct objects. However, if arrays are present at intermediate levels, Catapult would push the dimensions to the leaf fields and then, assemble the collections 'dimension-pushed' fields. This legacy layout scheme is unique to

Catapult and is aimed at exploiting the underlying QofR benefits. The RTL port of the struct type will be of flattened vector type.

```
struct dT {
    int a, b;
};
struct aT {
    dT dat[2];
};
Flattened logic vector old layout:
{dat[1].b, dat[0].b, dat[1].a, dat[0].a}
```

NOTE: For backwards compatibility, using the default values for *PRESERVE_STRUCTS* and *STRUCT_LAYOUT*, Catapult handles struct interface mapping as in previous releases.

```
directive set -PRESERVE_STRUCTS true # default is true
directive set -STRUCT_LAYOUT legacy # default value is legacy
```

In this release, Catapult provides a method to retain the C-style layout of struct objects at the interface that may be required for standardized view and IP assembly. Further, for ease of debugging, interface ports with corresponding struct/record types are desirable against a flattened vector form of the port. To aid these requirements, Catapult provides a new directive (*STRUCT_LAYOUT*) to enforce a *c_style* layout and netlist options to influence the format of the struct type ports. Note that these enhancements are applicable only the interface channel ports and/or SystemC interface Signal ports (*sc_in/sc_out*), containing the struct data type.

Based on the directive *STRUCT_LAYOUT* directive set to “*c_style*” (used in conjunction with the *PRESERVE_STRUCTS* directive set to true), Catapult will generate a new layout following the C-style scheme of packing fields and/or arrays. Using the new Netlist options, the datatype format of the struct-type ports in the RTL can be specified. In case of Verilog output, VerilogStructFormat option can be set to ‘packed’ (default value), ‘unpacked’ or ‘vector’. In case of VHDL output, VHDLStructFormat option can be set to ‘record’ (default value) or ‘vector’.

```
typedef struct packed {
    reg [31:0] a;
    reg[31:0] b;
} dT;
typedef struct packed {
    dT [1:0] dat;
} aT;
```

For an example of using the new options for structs on the interface, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Interface Struct-type mapping**).

New Timing Model for Reporting

In previous releases, Catapult used a single timing arc to model the delay through the library component.

In this release, the `ADVANCED_TIMING_MODEL` directive enables a more accurate estimation of timing in design that contains adder chains. This directive is particularly effective when used in combination with `X_PHD_SYNTHESIS`, since the latter improves the chance of adder chains to be generated by Catapult.

This new feature considers the fact that in an adder not all the output bits have the same delay. For example, in a carry chain adder, the first output bit has the delay of a full adder, the second has delay of twice the delay of a full adder and the Nth bit has the delay of N times the delay of a full adder.

When the feature is activated, Catapult will automatically generate timing models that can manage different arrival times. In this release, this new functionality is only available for reporting. It is not utilized by the scheduler at this time.

This functionality is only available in Catapult Ultra. Catapult HLS will continue to use the single timing arc from the library.

For an example of reporting using the new timing model, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Advanced Timing Model**).

Value Range Analysis Tool

This new tool allows you to perform value range analysis of `ac_fixed` variables in a C++ design that is being migrated from a floating-point representation to a fixed-point model. The core of this functionality is a runtime instrumentation feature added to the `ac_fixed` datatype that is available to Catapult users. This instrumentation captures the quantization and overflow errors occurring in an `ac_fixed` variable as it is being exercised by a C++ testbench. At the end of program execution, a summary report is printed (either to the standard output or a file) for each `ac_fixed` variable showing the value ranges, quantization error, call stack to the location of the declaration of the variable and a suggested refined bit-width based on the values observed.

For an example of using this app, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Value Range Analysis with AC_FIXED**).

Shared Memories using `ac_sync`

For `ac_channels`, Catapult links synchronization signals with the data that is synchronized (e.g. a data signal with `rdy/vld` handshaking signals). In previous releases, Catapult required the use of `ac_channels` to implement shared memories. Although this style continues to be supported, in this release, Catapult provides the new `ac_shared` style for shared memories which separates the data from the synchronization signals.

With the new `ac_sync` style for shared memories, the shared variable and the synchronization operations are separate objects. The explicit `sync_in()` and `sync_out()` operations of the `ac_sync` channel are blocking operations which allow you to synchronize concurrent processes. This approach allows you to associate an `ac_sync` operation with one or more sync objects. If that I/O can be scheduled in the same stage as the sync operation then `II=1` is possible allowing continuous operations.

This coding style utilizes the following guidelines:

- Only use `ac_shared` for the shared variable itself

- Only pass the contained object to other functions (eg. sub-blocks should not declare *ac_shared* objects on their interface)
- Declare the shared memory array as *ac_shared* (do not declare an array of *ac_shared* variables)
- Use a local reference to the contained object and make the reference as local as possible. Do not make this reference persistent (as a global or a class data member)

The following simple example creates a ping-pong memory following these guidelines:

```
#include <ac_sync.h>
#include <ac_shared.h>

void Block1(ac_sync &sync, Data_T mem[2*SIZE], ac_channel<Data_T> &in)
{
    static bool select;    // persistent, reset to false
    for ( int i = 0; i != SIZE; ++i )
        mem[(select*SIZE)+i] = in.read() * 0x1a5d;
    sync.sync_out(mem);
    select = !select;
}

void Block2(ac_sync &sync, Data_T mem[2*SIZE], ac_channel<Data_T> &out)
{
    static bool select;    // persistent, reset to false
    sync.sync_in(mem);
    for ( int i = SIZE; i; )
        out.write(mem[(select*SIZE)+(--i)] * 0x89f5);
    select = !select;
}

void Top(ac_channel<Data_T> &in, ac_channel<Data_T> &out)
{
    static ac_shared<Data_T[2*SIZE]> mem;    // persistent, not reset
    static bool d = ac::init_array<AC_VAL_DC>(mem, 2*SIZE);
    static ac_sync sync;                    // persistent
    Block1(sync, mem, in);
    Block2(sync, mem, out);
}
```

For multiple examples of using this new style for Shared Memories in various configurations, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Shared Memories Synchronized Access**).

AC Window 2.0

The new AC Window 2.0 classes provide a number of new features such as the ability to process multiple data inputs in parallel, more line buffer memory architectures to choose from as well as enhanced modularity. Because the function names and parameters are not backwards-compatible with the legacy AC Window library, you will need to replace the legacy AC Window instantiations in order to gain the benefits of this new feature.

For examples of using the new AC Window classes for image processing applications, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > IPL (ac_ipl) > ac_window_v2**).

DirectOutput Resource Type

Similar to the existing *DirectInput* resource type, this release introduces a *DirectOutput* resource type that occurs as soon as possible in the schedule and does not rely on being synchronized with other outputs. This enhancement would typically be used on such signals as pseudo-constant coefficients, status indicators or user debug ports that do not require being timed. This is also applicable to clock domain crossing in both C++ and SystemC.

Similar to *DirectInput*, you specify this resource type by setting the *MAP_TO_MODULE* directive with a value of "[*DirectOutput*]" on the port. You can also set the new *Direct* value on an interconnect and Catapult will automatically propagate the *DirectInput* and *DirectOutput* resource types to the ports connected to the interconnect. This approach can be helpful for top-down blocks that do not require timing on specific ports. For bottom-up blocks, you will need to explicitly set *DirectInput* and *DirectOutput* on the appropriate ports.

The following examples show how to set the *DirectOutput* and *Direct* values using the *MAP_TO_MODULE* directive. In the first example, the *Direct* resource type is set on the interconnect and Catapult propagates *DirectInput/DirectOutput* to the appropriately connected module ports.

```
directive set /fir/coeffs0.d.c:rsc -MAP_TO_MODULE {[Direct]}
```

This example explicitly sets the *DirectInput* and *DirectOutput* resource type on connected module ports.

```
directive set /fir/fir_filter<0>/coeffs.c:rsc -MAP_TO_MODULE {[DirectInput]}
directive set /fir/load_coeffs/coeff0.c:rsc -MAP_TO_MODULE {[DirectOutput]}
```

You can also set this new resource type in your source code using the *hls_resource* pragma. Since the pragma applies to an interface variable, you must define the pragma just prior to the function in your source code.

```
#pragma hls_resource coeff0.c:rsc variables="coeff0" map_to_module="[DirectOutput]"
void load_coeffs( ac_channel<ac_int<1,false> > &load, coeffT &coeff0 ...
```

For an example of specifying the *DirectOutput* Interface type using directives or pragmas, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > DirectOutputUsage**).

Design Analyzer Improvements

The following list summarizes the improvements to Design Analyzer in this release:

- Schematic Viewer improvements
- [Value Range Analysis tool](#)

Library Changes

The following libraries/components were added in this release:

- **Lattice FIFOs** library (Lattice_FIFO) contains three new FIFO components. The asynchronous FIFO (FIFO_async) and synchronous FIFO (FIFO_sync) contain instantiations of the *lscf_fifo* macro plus some control logic. The only difference between these components is the reset type. The implementation of the Depth of 1 FIFO (*FIFO_depth1*) is strictly RTL without any *lscf_fifo* instantiations. It has a fixed depth of 1.
- **mgc_sqr** component added to base library template. At this time, this component may not be utilized by the scheduler or optimizations.
- **ccs_pipe_regfile_cdc** added to *ccs_ioport* library. This new component is identical to the register-based FIFO component (*ccs_pipe_regfile*) except it utilizes separate clock, enable, async reset, and sync reset ports on the Data Input and Data Output of the pipe component.
- **DesignWare Floating Point Libraries** removed the *prevent_OTF* variable on components. In previous releases, this variable prevented DW FP components from running On-The-Fly characterization. In this release, this variable was removed from the library template.

If your DesignWare Floating Point library was created using a previous release, you can update (or remove) the *prevent_OTF* variable on each DesignWare library component variable so it will match the new library template. You can open your DesignWare Floating Point library in Library Builder, run the following command and save the library.

```
library set /LIBS/*/MODS/ccs_dw_*/VARS/prevent_OTF/VALUE -match glob "false"
```

- **Xilinx Versal** library added support for the Precision flow within Catapult.

New CFormal Assert App (Beta)

In previous releases, the only way to determine where an assert would be triggered required dynamic simulation.

In this release, a beta version of the Cformal Assert app is available within Catapult. The app runs Catapult Formal to formally determine whether an assert() and/or cover() statement will trigger.

For an example of using this app, refer to the example toolkit at \$MGC_HOME/shared/examples/methodology/assertion_synthesis or access it via the Start Page in the Catapult GUI (Examples > Methodology > Assertion Synthesis).

Bus Slave Generator Flow (Beta)

Catapult is commonly used to create accelerator blocks that sit on a system bus and are configured by a processor. For example, a video filter may implement multiple display resolutions and filtering modes. The C++ design for the algorithm can be synthesized easily with Catapult. But the resulting RTL needs to be integrated into the larger system design with a register interface address map that allows the CPU to “program” the operating mode of the filter – changing resolution and/or filter mode.

This new flow automatically wraps your HLS design block with a bus slave interface for configuration. The Bus Slave Generation flow handles generation of the HDL that implements the registers, control logic and address decode along with the a top-level design that wraps the slave register interface(s) with the post-HLS design RTL.

Note: This feature is provided as an Early-Access Beta feature for this release.

For an example of using this app, refer to the example toolkit via the Start Page in the Catapult GUI (**Examples > Methodology > Bus Slave Generator**).

Punch ac::probe() Signals to Top-Level (Beta)

In this release, the ac:probe() function was enhanced to push the probe through hierarchy to the top-level interface. To accomplish this enhancement, an optional third argument (shown below) was added to the ac::probe() function. For backwards compatibility, the default value for the third argument (ac::ProbePunchNone) retains the functionality of previous releases.

```
ac::probe(string probe_name, T* probe_expression, punch = ac::ProbePunchNone)
```

The third argument (punch) can be one of the following enum values:

- ac::ProbePunchNone : (default) No port punching of ac::probe signal
- ac::ProbePunch : punch the probe to the top level .
- ac::ProbePunchReg : Register the probe and punch it to the top.

When Catapult punches the probe to the top-level interface, the valid and data signals are both punched so you know when the data is valid. If a probe is used in a hierarchical block, and the block is used more than once, then the name is extended with ‘_<##>’ to unify the name.

The current SCVerify behavior is unchanged. Any probe data/valid port-punched signals show up in a new *Interface_Probe_Signals* section in the waveform display in simulation.

Given the following ac::probe call, Catapult creates an ac::probe called AVAL2 that passes the value of tVal1 and punches the data and valid signals to the top-level interface:

```
#include "top.h"
#include "ac_probe.h"

#pragma hls_design top
void top(ac_channel<uc> &Tin, ac_channel<uc> &Tout) {
    while (Tin.available(2)){
        uc tVal1 = Tin.read();
        ac::probe("AVAL2", tVal1, ac::ProbePunch);
        Tout.write(tVal1 + Tin.read());
    }
}
```

The resultings top-level design will include two interface signals for the port-punched probe:

```
output AVAL2_acprobe_vld;
output [7:0] AVAL2_acprobe_dat;
```

NOTE: This functionality is not valid for CCOREs.

Changes to Directives/Options/Commands

Directive Changes

The following list describes the changes to the Catapult directives.

- **ADVANCED_TIMING_MODEL** (*new directive*) controls the timing model used by Catapult. In previous releases, each library component contained a single timing arc (any input to any output). In this release, Catapult begins the introduction of a more accurate timing model containing point-to-point timing arcs. For the initial release, this new model is only available during reporting. Refer to New Timing Model for Reporting for a summary of this functionality and links to design examples.
- **ENABLE_10_1_COMPATIBLE_ARRAY_NAMES** (removed). In the v10.1 release, this limited-time directive was added to provide backwards compatibility for an array naming change. At the time of it's introduction, it was also announced that it would be deprecated in the v10.4 release. It was removed from Catapult in this release.

- **STAGE_IO_CNS** (changed values). In previous releases, this directive used boolean values.

In this release, the STAGE_IO_CNS directive can be one of three allowed values: *none* (formerly false), *soft* (formerly true), or *hard* (new value which informs Catapult to retain all 'soft' IO constraints). For backwards compatibility, Catapult automatically maps the previous boolean values to the new values (true == soft; false == none).

- **STRUCT_LAYOUT** (*new directive*). In previous releases, Catapult mapped struct types on the interface to a single vector. In this release, Catapult added this directive to allow you to control the signal type of structs on the interface. This directive is only valid when PRESERVE_STRUCTS is true

and the port is *ac_channel*, *sc_in*, *sc_out*, *sc_fifo*, *Connection::In*, or *Connections::Out*. This directive does not affect CCOREs.

Using the default value (*legacy*), Catapult is backwards compatible with previous releases. It will order the struct interface type into a single vector that is indexed by each struct element. If this directive is set to *c_style*, Catapult will map the struct on the interface to a single object and implement it as a VHDL Record or Verilog struct in the RTL netlist. This style allows for easier mapping of interface ports to C/C++/SystemC testbenches.

Refer to Improved Struct Type Interface Mapping for a summary of this functionality and links to design examples.

- **USE_PIPE_DEBUG** (new directive) specifies whether to implement interconnect pipes with *ccs_pipe* (default) or the debug component version (*ccs_pipe_debug*) which routes the pipe sync signals to the top-level interface. In previous releases, you would have to explicitly use the *MAP_TO_MODULE* directives to individually map the interconnect pipes.

Option Changes

The following list describes the changes to the Catapult options (Tools > Set Options...):

- **Bus Slave Gen flow** This [new flow](#) is in BETA for this release.
- **Enable checks on Assertion statements** (*Cformal Assert Flow*). This new option allows you to control whether the Catapult Formal Assert tool evaluates assert statements. This Flow is in BETA for this release.
- **Enable checks of cover statements** (*CFormal Assert Flow*). This new option allows you to control whether the Catapult Formal Assert app evaluates cover() statements. This Flow is in BETA for this release.
- **Enable trace/waveform generation for covered cover statements** (*CFormal Assert Flow*). This new option allows you to control whether the Catapult Formal Assert app generates waveforms for covered cover() statements. This Flow is in BETA for this release.
- **Waveform database format**. *qwave* or *fsdb* (*CFormal Assert Flow*) This new option allows you to specify the format of the generated wave file. This Flow is in BETA for this release.
- **Specify Global variables (set_global)** (*Cformal Assert, SLEC, and Stall Flows*). This option was removed from these three flows in this release.
- **Insert OBS score threshold** (*Low Power flow*) This new option allows you to set the effort level for the Observability logic optimization to medium or high (default). This value is a float between 0.0 (default) and 1.0. This option passes the *-threshold* switch to the *insert_observability_logic* command in PowerPro.
- **Insert STB_C score threshold** (*Low Power flow*) This new option allows you to set the threshold for including Stability score for combinational logic. This value is a float between 0.0 (default) and 1.0. This option passes the *-threshold* switch to the *insert_stability_logic -type C* command in PowerPro.

- **Insert STB_S score threshold**(*Low Power flow*) This new option allows you to set the threshold for including Stability score for sequential logic. This value is a float between 0.0 (default) and 1.0. This option passes the *-threshold* switch to the *insert_stability_logic -type S* command in PowerPro.
- **Enables PowerPro vectorless flow (no switching activity files)** (*Low Power flow*) This new option allows users to utilize the vectorless flow within the integrated version of PowerPro. By default, Catapult Ultra reads the switching activity files (eg. QWave, FSDB, or SAIF) for power reporting and optimization. With the vectorless flow, no simulation data is read and PowerPro estimates power usage. If you enable this option and specify switching files, then Catapult will not execute the vectorless flow.
- **Generate makefiles at stages** (*SCVerify Flow*) In this release, the *memories* and *architect* stages were added to the list of stages where SCVerify generates Makefiles.
- **Compile and elaborate design only** (*UVM Flow*) If enabled, this option informs the simulator to compile the design and testbench, load the design into the simulator and run the simulation for 0ns. This approach elaborates the design in the simulator so you can validate there are no errors prior to applying the test vectors.
- **Location of GDB install** (*VSCode Flow*) This new option allows you to override the filepath to the gdb executable used by VSCode. When using this flow, it is recommended that you use the *gdb* version included with your platform install instead of the version that ships with Catapult.
- **Optimization Analytics** (*Message Options*) This new option provides additional information to the transcript on optimizations.
- **Use Old Scheduler** (*Architectural Options*) This option was removed. If you attempt to set this option, Catapult reports an error. The Old Scheduler was completely removed from Catapult in a prior release.
- **Verilog Struct Format** (*output option*). This new option controls how Catapult writes *struct* interface types in the RTL netlist. The values can be packed (default), unpacked, or vector. This option is only considered when PRESERVE_STRUCTS is *true*.
- **VHDL Struct Format** (*output option*). This new option controls whether Catapult writes *struct* interface types in the RTL netlist. The values can be record (default) or vector. This option is only considered when PRESERVE_STRUCTS is *true*.
- **ClockGatingMinWidth** (*Prototyping Flow*) The default value for this option changed from 4 to 0.

Command Changes

In previous releases, Catapult could generate netlists in the Verilog and VHDL formats. In this release, Catapult can also generate netlists in the SystemVerilog format in order to support struct interface mapping.

Corrected Problems

The following customer reported problems were fixed in this release.

- CAT-34228 Beta Struct layout option creates simulation mismatch
- CAT-34169 Value Range Analysis variable names are not displayed
- CAT-34168 Value Range Analysis crash
- CAT-34123 ac_ieee_float set_data() is missing in Raw Data Access Method
- CAT-34020 Mismatch in port interconnections for BUP v/s TD flow
- CAT-33999 Wrong debug signal names in ccs_pipe_debug
- CAT-33970 Class-base Hierarchical w/CCore : internal assertion sif_if_clock.css
- CAT-33953 Flow2 Wrapper enable and document limitations
- CAT-33895 CFormalAssert : GUI debug menu does not match CFormalAssert.rpt
- CAT-33787 Support for Multiple template in CCS_DESIGN macro
- CAT-33768 sif assert sif_flows_api.cxx : At schedule class based hierarchical design when synthesized top down
- CAT-33767 Wrong naming of ports
- CAT-33765 Keep plain Verilog for VerilogStructFormat=vector
- CAT-33763 Parallelogram Timing : Design Slack Regressed to negative value
- CAT-33686 No difference in generated rtl for Verilog netlister options for packed and unpacked
- CAT-33669 implement PRESERVE_STRUCTS=false solution for SystemC interface
- CAT-33639 CFormal name inconsistency causes failure to run
- CAT-33604 CFormal Makefile.*.* clean rule missing the write_db cleanup for stall, slec
- CAT-33597 Lint warnings on signed arithmetic
- CAT-33582 Spyglass reports X assignment error on ccs_fifo_wait_core_v5
- CAT-33567 ASSERT-1 Error using Matchlib
- CAT-33562 Assertion failure in OTF prototyping for "mgc_add_msb" component
- CAT-33525 Pragma List: Not consistent between the HTML and PDF
- CAT-33485 directive WORD_WIDTH doesn't break internal register array.

- CAT-33380 Support non-static data member initializers from C++11
- CAT-33379 non-static data member initializers causing internal assertion in compile phase
- CAT-33360 Allocation after PHD is incorrect
- CAT-33356 An extra input is generated for CCORE
- CAT-33318 Support creation of AXI-Lite register bank for Telco applications - Top Wrap Gen
- CAT-33317 Support creation of AXI-Lite register bank for Telco applications - CMacro Gen
- CAT-33315 Support creation of AXI-Lite register bank for Telco applications - VHDL CSR Gen
- CAT-33314 Support creation of AXI-Lite register bank for Telco applications - CSV read
- CAT-33297 Mapped Bottom-up CCORE flow failure with unmatched number of ports
- CAT-33289 Support for DW components in the OTF flow
- CAT-33211 Tool crashes when templated Class having a non-templated member function defined as CCORE
- CAT-33188 Put optimize-place after LEF check in app_or.oastcl.tcl to allow for Oasys prototyping
- CAT-33143 UVM creates issue in memory loading on predictor side when internal array mapped to external memory
- CAT-33140 UVM creates issue in memory loading on predictor side
- CAT-33084 Difference in Qofr values between hls_pipeline_init_interval pragma and GUI option
- CAT-33083 hls_pipeline_init_interval pragma not working correctly
- CAT-33031 UVM simulation throws compilation error for external memory mapping
- CAT-33030 UVM flow should support memories on the design interface
- CAT-32981 crash during architect when MUX_OPT_SELECTOR set to high or medium
- CAT-32974 Syncing Verdi version in release notes of Catapult and PowerPro
- CAT-32946 Cformal SLEC Makefile.formal.v location missing Info: message for dut_c.v1 . Warning message has eclipsed it.
- CAT-32926 Remove dependency to Questa when running PowerPro
- CAT-32924 Catapult creates misbehaving RTL when using ac_int reverse method

- CAT-32923 enable on/off pragma in the generated OVL module instances when using cover()
- CAT-32834 Add mgc_mux(2,16384) to std cell base lib template
- CAT-32788 Library contents not displayed when using new template
- CAT-32764 Class interface function shows-up separately in GUI when using pipeline pragma
- CAT-32756 catapult.log for Stall test has Falsified/Unresolved counts reversed. slec.log is correct.
- CAT-32753 Create C++ sim models for DW_fpflt2i, DW_fpflt
- CAT-32699 Need directive to treat ccs_pipe_debug as default component for internal channels
- CAT-32593 IO_READs being pushed into pipeline. TRANSACTION_SIGNAL_DONE effects the scheduling of IO_READS()
- CAT-32552 Catapult Ultra launched PowerPro is missing some clock gating moves
- CAT-32536 SIF assert in go assembly
- CAT-32463 How to schedule 2 read() in the same cycle
- CAT-32331 Enhance AXI-Stream interfaces for Telco applications
- CAT-32243 Support creation of AXI-Lite register bank for Telco applications
- CAT-32154 CCOREs in templated classes causing bad RTL and scverify to fail
- CAT-32146 UVM flow agents do_wait_for_reset() task is incorrect
- CAT-32077 License Daemon Requirements document for CCOV 2022.2 is old
- CAT-31991 Document nb_read_join
- CAT-31525 Catapult taking large runtime in allocation phase in a design with 4 nested unroll loops
- CAT-31454 memory IO scheduling with ac_sync, documentation update required
- CAT-31448 BOM crashes Catapult on customer design
- CAT-31399 Productize analyze stage hierarchy GUI
- CAT-31368 Add CFormalAssert testing to toolkit Methodology/Assert
- CAT-30554 Design Analyzer not showing all hierarchical blocks
- CAT-30473 UVM generation fails with array of sc_ins

- CAT-30337 Line buffer implementation option in AC Window
- CAT-30317 X from invalid transaction(vld=0) is registered
- CAT-30137 Assertion at schedule state when a pipeline stall mode specified is flush
- CAT-30017 crash using ac_shared on hierarchy interface
- CAT-29540 Unused direct input errors out in class-based hierarchical bottom-up flow
- CAT-29459 Requesting support for Lattice FPGA – FIFOS
- CAT-29425 Document custom scripting for prototyping flow
- CAT-29083 Unexpected toggles at the input of leaf CCORE
- CAT-27336 Prototyping flow only able to accept explicit list for excluded cells
- CAT-27319 provide version of ac_probe() that port-punches the signal to the interface
- CAT-27179 Design Analyzer crashes when opening up failed schedule
- CAT-26498 Preserve C++ struct's fields order in RTL interface
- CAT-26155 ROM Prototyping failure due to dependency on ICG cells
- CAT-25180 Power optimized netlist fails SCVerify verification
- CAT-13357 Request for direct_output interface
- CAT-11922 Provide options to control LP optimization
- CAT-11168 Catapult output RTL could maintain struct data types
- CAT-10939 Catapult Setup Wizard will crash if the saved license config is wrong
- CAT-10167 mgc_pipe_regfile_cdc needs max_delay SDC constraint for downstream synthesis
- CAT-7975 Enhance Catapult to handle libraries with different library units to avoid Error: Incompatible library units

Known Problems and Workarounds

The following sections describe known problems in this release and provide workarounds when they exist.

Color Space Conversion example uses unavailable library (2025.1)

In the Color Space Conversion toolkit example (Examples>IPL (ac_ip1)>Color Space Conversion), the HLS technology libraries specified in the Tcl scripts are not shipped with Catapult. Please use a different library to run the different flows in the toolkit example.

Mapped Bottom-Up flow in Beta

The "Mapped Bottom-up Flow" is currently in Beta. It is recommended that customers use the regular Bottom-up flow

- In the **regular Bottom-Up flow**, Catapult builds the bottom-up (BU) blocks during the compile stage. During assembly, you can map the BU block to a previously synthesized block. This is the recommended flow.
- In the **mapped Bottom-Up flow**, Catapult only builds the interface to the bottom-up block and does not build the body of the block. This approach helps both in terms of capacity and reduced runtime. But, it is paramount that the interface of the mapped BU block is recreated in the identical manner as when it was synthesized earlier standalone. Along with other interface properties, the datamode is very critical to be correct.

```
solution design set dct -mapped
```

The mapped Bottom-Up flow is not a production quality feature yet and it not the default recommended Bottom-up flow for users.

Incomplete Floating-Point, Low-Power VHDL sim model in DesignWare

Because some VHDL simulation models for the Low-Power DesignWare components do not produce any valid outputs, the VHDL netlist dependencies for all DW_LP_PIPED_XXX components in the optional DesignWare Catapult library were removed to prevent users from utilizing any incomplete models.

As a result, Catapult will still target these components during HLS but you will not be able to use the generated VHDL RTL netlist in downstream flows. Until these Designware models is fixed, if your design

uses one of the previously mentioned DW components in the Catapult-generated netlist, you can work around this issue by using the Verilog RTL netlists. (CAT

SCCOM error with MSIM_AC_TYPES enabled

The MSIM_AC_TYPES option in the QuestaSIM flow may cause some designs to error when compiling with QuestaSIM's SystemC compiler *sccom*.

The issue may occur using QuestaSIM versions 2023.1 and later releases. All of these QuestaSim versions use *gcc* 10.3 under the hood of *sccom*. If you encounter this issue, you will get the following error message during *sccom* compilation:

```
# Compilation terminated.  
# ** Error: (sccom-6143) Creation of debug database failed.
```

To resolve this issue, you can disable the MSIM_AC_TYPES QuestaSIM flow option. Disabling the option will not impact simulation. It will only the debug view of the types for ports in the wave window.

```
options set Flows/QuestaSIM/MSIM_AC_TYPES true
```

Running Design Analyzer on Unsupported RHEL releases

The 2023.2_1 release of Catapult supports the RHEL 7.9 release. If you need to run on an earlier release of RHEL 7, then you should be able to launch the 2023.2_1 Catapult DA release using the following recipe.

```
#!/bin/sh  
ccs_install=/path/to/install/ccs_2023.2_1/Mgc_home  
lib_install=/path/to/install/extra  
lib_dir="$lib_install/usr/lib64"  
if [ ! -d "$lib_dir" ]; then  
  ( mkdir -p $lib_install \  
    && cd $lib_install \  
    && wget https://vault.centos.org/7.6.1810/os/x86_64/Packages/freetype-2.8-  
12.el7.x86_64.rpm \  
    && wget https://vault.centos.org/7.6.1810/os/x86_64/Packages/  
harfbuzz-1.7.5-2.el7.x86_64.rpm \  
    && rpm2cpio freetype-2.8-12.el7.x86_64.rpm | cpio -idmv \  
    && rpm2cpio harfbuzz-1.7.5-2.el7.x86_64.rpm | cpio -idmv )  
fi  
env LD_LIBRARY_PATH="$lib_dir${LD_LIBRARY_PATH:+:}$LD_LIBRARY_PATH" "$ccs_in-  
stall/bin/catapult" "$@"
```

Update the two variables at the top to the desired absolute path locations and `chmod +x` the script so you can execute it. Then you can use the script to configure and invoke Catapult passing any arguments.

This recipe was confirmed using CentOS 7.1, as well as 7.5.

VHDL memory-based pipe may not synthesize in Design Compiler

Design Compiler may not synthesize the VHDL implementations for the *ccs_pipe_regfile* and *mgc_pipe_regfile* register-based pipe interconnects but will synthesize without issues in other RTL synthesis tools. The Verilog implementations for both of these components work as expected in Design Compiler.

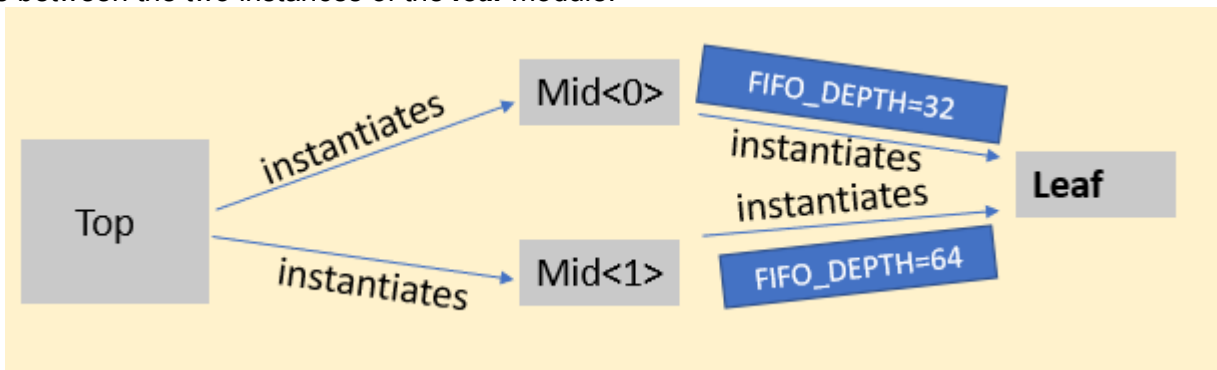
Setting different FIFO_DEPTH values on multiple instances design

In Catapult 2022.1 and prior releases, you could set different FIFO_DEPTH constraints for the independent FIFOs connecting different instances of the same block.

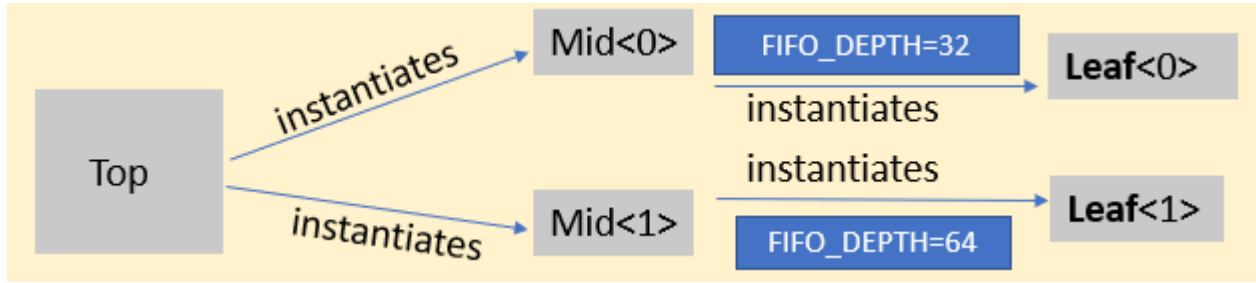
In 2023.1, Catapult will issue an error going forward which is as follows.

```
Filename<linenumber>: Interconnect bus "CNS_PATH:CNS_NAME" has incompatible specification for 'FIFO_DEPTH'. It conflicts with one or more of the following objects ?List of potentially conflicting objects with FIFO_DEPTH constraints
```

The following image shows the configuration that generates the error message because the `fifo_depth` values conflicts between the two instances of the *leaf* module.



The workaround for this issue is to unify the instance hierarchy so that there is one to one mapping between instance and module. As shown in the following illustration, you could use template recursion to unify every instantiation of the *leaf* module. This implementation can be synthesized without the error because the *leaf* module instance path is unique.



ASIC library template for mgc_div in 2022.1 may cause flow issues

ASIC libraries characterized with Catapult 2021.1, 2021.1_1, or 2022.1 use a mgc_div module that relies on a Synopsys DesignWare implementation. The mgc_rem and mgc_mod modules were also removed from the library because the DW_div supports these modes. These changes caused some issues with downstream flows. To revert back to the three older modules, you need to refresh your Catapult library using the 2022.1_1 or later release of Catapult. This process will use the new ASIC base library template after deleting the mgc_div.

To get back to the standard mgc_div, mgc_rem, and mgc_mod modules:

1. Open the library with 2022.1_1 (or newer) version of Library Builder
2. Remove the current mgc_div module
3. Refresh the library
4. Characterize the refreshed modules

The following script shows the command to accomplish this process:

```
#Example refresh run script:
set lib nangate-45nm_beh
if { [library get /LIBS/$lib -return leaf -checkpath 0] != $lib } {
  puts "Error: Library '$lib' not loaded"; return
}
library remove /LIBS/$lib/MODS/mgc_div
library refresh -library $lib -update 1
library characterize $lib
```

Memory Generator does not merge multiple read or write ports when memory uses different read and write clocks

If a RAM has multiple read ports or multiple write ports, the multiple read ports should be merged into a single port (with a count property) if the read ports are all similar (i.e. have the same clock and same type of control signals), similarly the multiple write ports should be merged into a single write port if the write ports are similar. This merging is necessary for Catapult to schedule multiple operations in parallel. This merging is not performed (but should be) when all of the read ports share a common read clock, and all of the write ports share a common write clock that is not the same as the read clock.

Memories in Bottom-Up Blocks may not be blackboxed in Low Power flow

The sample RAM models shipped with Catapult have empty implementations for synthesis. In other words they use *pragma translate off/on* to turn off the simulation behavior during synthesis.

Normally, Catapult LP will automatically find such RAMs in the first two levels of hierarchy and mark them as black-box memories prior to doing power optimizations. This will prevent power optimization from making moves that might cause a simulation mismatch.

However, if the design has internal memories deep within a bottom-up hierarchical design, Catapult may not find them and mark them as black-box. You can determine if your design has such deep RAMs by looking for either of the messages shown below in the Catapult LP transcript window:

```
# /apps/Mgc_home/pkgs/siflibs/ram_sync_separateRW_be_generic.vhd(98): Connect-  
ing 'z' to unconnected port 'data_out' in module 'ram_sync_separateRW_be'.  
(CPT-DC6)  
# Warning: Hierarchy 'ram_sync_separateRW_be' (/apps/Mgc_home/pkgs/siflibs/  
ram_sync_separateRW_be_generic.vhd:98) is empty. (CDB-CEH)
```

If you find such messages in your transcript, you can add the command

```
create_black_box -module <ram modulename>
```

in the LowPower_startup.tcl script in your current working directory. For assistance with this issue, contact customer support.

SystemC Direct pinassocs use hierarchical name

Catapult allows you to specify *Direct* pinassocs to automatically route a pin to the top level of the design. For SystemC designs, if you specify a "Direct" pinassoc, Catapult generates a port name based on the

hierarchical resource name of the *direct* pinassoc. As a result, the name of the *direct* pinassoc does not match an existing port on the top level interface so the *direct* pinassoc is not connected to the top level interface. Global pinassocs work correctly for SystemC designs (assuming that the global pin name already exists on the top level interface).

SCVerify fails to detect simulation deadlock in bottom-up hierarchy designs

SCVerify has the ability to detect simulation deadlocks in hierarchical designs by monitoring the idle state of each block along with the primary inputs and outputs. This feature works correctly for top-down hierarchical designs but may fail to detect a deadlock when the same design is synthesized bottom-up. This is because SCVerify loses visibility into the idle state of the subblocks. The result is that simulation will hang. If you are using SCVerify with a bottom-up design and you encounter a simulation hang it may be this issue. Try synthesizing the design top-down and simulation. If you get the deadlock message, adjust the size of the internal FIFO components between the blocks and then apply that same FIFO_DEPTH constraint in you bottom-up synthesis.

Memories with EXTERNAL pinassoc can not be shared

If any pin on a memory component is defined with an *external* pinassoc, then Catapult externalizes the entire memory from the design. This action only occurs for memories with *external* pinassocs. Catapult creates a port component on the top level interface to connect the external memory to the design. Memories with an external pinassoc can not be used as a shared memory but will work as expected for non-shared memories.

SystemC - Per-file Compile Flags not Passed to SCVerify

Catapult provides two methods for setting compilation flags (such as “-DUSE_MY_TYPE”) to use for compiling a design both for synthesis and verification. The first is to use the Catapult option “option set Input CompilerFlags” and the second is to add the flags to each input file as they are added to the project using the “-args” switch to the “solution file add” command. For non-SystemC based designs both methods will give the desired results. However, for SystemC based designs the per-file “-args” switch is ignore for the SCVerify verification flow. The workaround is to use the CompilerFlags option only.

Multiple Drivers in Modular I/Os

If you use the modular I/O provided with Catapult, signals with multiple drivers will be recognized and optimized in the RTL output. However, multiply-driven signals in other more complex custom modular I/O will return an error.

For more information on the supported modular I/Os, see the "Inferring Optimized Modular I/Os" section of the System C User's Guide.

SystemC SCVerify Limitation

The SCVerify flow does not work with designs that use a top-level I/O built from an interface containing an SC_THREAD, SC_CTHREAD or SC_METHOD. The primary I/O should be built from signals, classes containing modular I/O functions, sc_fifo_in, or sc_fifo_out. There should be no behavioral code in the interface of the top design.

Except for clock or reset signals added automatically, Catapult expects an explicit interface port (sc_in/out, sc_fifo_in/out) so each RTL port can be connected properly. A clock or reset transactor module is added automatically in the SCVerify wrapper when they are missing (for example, in sc_fifo_in/out interfaces).

ROM Inferencing

In some cases, Catapult will not be able to infer ROMs unless they are declared "static const". All constant arrays that would be mapped to ROMs need to be declared in the following way to avoid this issue:

```
static const ac_int<8> my_rom[4] = {1,2,3,4};
```

RAM Selection Algorithm May Not Choose Smallest RAM

The default RAM selection criteria will pick the smallest size RAM component available that can fit a given resource (RAM size = RAM width x RAM depth). However, the selection algorithm may not choose the smallest one when choosing between RAMs that are different sizes, but have the same address bit width. For example, if the resource dimension is 32x128, and the two RAMs available are RAM1(32x256) and RAM2(32x200), it is possible that Catapult will choose the larger RAM.

Compact Gantt Chart is Slow for Large Designs

The complexity of the placement algorithm used for the Compact mode Gantt chart requires more time than the Expanded mode. For large designs, generation of the compact Gantt chart can take a very long time. The performance of the compact Gantt chart will be improved in future releases of Catapult.

Workaround: For large designs, avoid using the Compact mode feature. Be sure that the default mode is set to Expanded (the factory default). Use the following command to set the default mode:

```
options set ScheduleViewer DefaultScheduleView Expanded
```

Direct Input Mapping Only Works for One Direct Input

The "[direct input]" mapping for inputs is not working as expected if you have a design with two or more inputs mapped to direct input. A soft constraint is added between all inputs mapped to direct input, which causes

them to all be read in the same pipeline stage.

The workaround, and possibly the long-term solution, is to set `STAGE_IO_CNS` to false, which removes all soft I/O constraints from the design. You then must use the cycle command on all inputs and outputs to select what cycle to schedule them.

The “!” Operator is Not Supported with `sc_fixed` Datatypes

The SystemC not operator (!) when used with a variable of type `sc_fixed` or `sc_ufixed` will cause the variable to be converted to type double, which in turn may cause incorrect logic to be generated.

Area Over-Estimation for Designs with Small Muxes

Catapult will over-estimate the area for designs that contain a large number of small muxes. This is purely a reporting issue and will not affect the real area of the design.

Longer Schedule for Loops with Multiple Exits

In some rare cases, an I/O write will be scheduled at least one clock cycle after a loop if the loop has multiple exit points that might affect the value of the I/O write. Scheduling the extra clock cycle addresses a potential issue with pipeline ramp-down when the loop is pipelined. A future enhancement will allow the I/O write to be scheduled in the same C-Step as the loop exit.

SCVerify Flow: Avoid Using Output Memories When Pipelining

The detection of correct outputs when simulating the untimed C++ function along with the timed Catapult netlist requires using a global synchronization signal `out_sync` to indicate that all of the outputs are updated and stable. When memory writes are scheduled in pipelined loops, the writes may occur in a different stage than the other outputs and thus the `out_sync` signal will capture the next (or the previous) contents of the memory rather than the correct contents.

Third-Party Information

The following HTML page provides information on third-party software that may be included in the Catapult product, including any additional license terms. This document is located at the following location within the Catapult install tree:

```
$MGC_HOME/shared/pkgs/sif_docs/legal/Catapult_OSS.html
```